
ParEVO: Synthesizing Code for Irregular Data: High-Performance Parallelism through Agentic Evolution

Liu Yang^{*1} Zeyu Nie^{*1} Andrew Liu¹ Ruomu Zou¹ Deniz Altinbüken² Amir Yazdanbakhsh²
Quanquan C. Liu¹

Abstract

Parallelizing code for **irregular data structures** (sparse graphs, unbalanced trees, non-uniform meshes) is notoriously hard, and current LLMs fail catastrophically on such tasks, generating code riddled with race conditions, deadlocks, and poor scaling. We address this with **ParEVO**, a framework for synthesizing high-performance parallel algorithms for irregular data, built on three contributions: the **Parlay-Instruct Corpus** of 13,820 tasks generated via a Critic-Refine” pipeline that filters for empirically performant uses of Work-Span primitives; specialized **DeepSeek**, **Qwen**, and **Gemini** models fine-tuned to the semantics of the ParlayLib library; and an **Evolutionary Coding Agent (ECA)** that repairs the last mile” of correctness using compiler and profiler feedback. On the ParEval benchmark, ParEVO achieves an average $107\times$ **speedup** and a $13.6\times$ **speedup** on highly complex irregular graph problems, outperforming commercial models like GPT-5-Thinking and Gemini-3-Pro, while matching expert *human-written* baselines and reaching up to a $4.1\times$ **speedup** on kernels such as Maximal Independent Set. This demonstrates that AI-driven agents can effectively navigate the complex landscape of high-performance computing. Source code and datasets are available at <https://github.com/WildAlg/ParEVO> (ParEVO, 2026a).

¹Department of Computer Science, Yale University, New Haven, CT, USA ²Google DeepMind, Mountain View, CA, USA. Correspondence to: Quanquan C. Liu, Liu Yang, Zeyu Nie <quanquan.liu@yale.edu, liu.yang.ly337@yale.edu, zeyu.nie@yale.edu>.

1. Introduction

The breakdown of Dennard scaling and the subsequent stagnation of single-core frequency scaling have fundamentally shifted the computing paradigm. Performance improvements in modern software are now almost exclusively driven by parallelism, whether through multi-core CPUs, GPUs, or distributed clusters (Sahu et al., 2019). While “regular” parallelism (e.g., dense matrix multiplication) is well-understood and supported by mature libraries, **irregular parallelism** remains a grand challenge in High-Performance Computing (HPC).

Irregular algorithms, which operate on graph structures, sparse matrices, or adaptive meshes, are characterized by unpredictable memory access patterns and dynamic work distribution. In these regimes, the computational cost of processing a node or element depends on runtime data, making static load balancing ineffective. Writing efficient code for these problems requires sophisticated techniques like work-stealing, dynamic scheduling, and lock-free synchronization (Nichols et al., 2024).

Current Large Language Models (LLMs) struggle profoundly with this domain. Trained primarily on sequential Python or standard C++ code from GitHub, they exhibit strong “sequential bias.” When asked to parallelize a graph traversal, they often attempt to wrap a standard Breadth-First Search (BFS) in a naive parallel loop (`#pragma omp parallel for`), ignoring the race conditions inherent in updating the ‘visited’ array. Alternatively, they may introduce coarse-grained locks that serialize execution, rendering the parallel code slower than its sequential counterpart (Kambhampati et al., 2024).

We argue that the solution lies not in teaching LLMs to write low-level threading primitives (like ‘`pthread`’ or ‘`std::thread`’), which are error-prone and hard to compose, but in leveraging high-level algorithmic primitives. **ParlayLib** (Blelloch et al., 2020) provides a suite of such primitives (e.g., ‘`filter`’, ‘`pack`’, ‘`scan`’, ‘`sort`’, ‘`reduce`’) that abstract away the complexities of scheduler management. By training LLMs to map natural language intent to these primitives, we can generate code that is *correct by construction*.

and mathematically provable to scale.

To this end, we introduce **ParEVO**, an end-to-end system for synthesizing high-performance parallel code. We detail the following main contributions:

- **Data-Centric Synthesis:** We introduce the *Parlay-Instruct* corpus, a dataset of 13,820 parallel coding tasks. Unlike previous datasets scraped from GitHub (which often contain broken code), our data is synthesized via a “Teacher-Student” pipeline and verified against a ground-truth compiler oracle. We provide a novel performance dataset generation technique focused on graph problems curated from the selection of well-known programming competitions curated by the online-judge DMOJ (DMOJ Developers, 2024).
- **DeepSeek-Parlay, Qwen-Parlay, Qwen-Rust, and Gemini-2.5-Parlay¹:** We release a fine-tuned 6.7B parameter Deepseek model for C++ (ParEVO, 2026b), two fine-tuned 30B parameter Qwen3 models—one for C++ (ParEVO, 2026) and one for Rust (ParEVO, 2026)—and a Gemini-2.5-Pro model fine-tuned for C++. These models outperform some larger closed-source and open-source models on parallel reasoning tasks by internalizing the data structures, semantics, primitives, and algorithms of the state-of-the-art ParlayLib library (Blelloch et al., 2020) and safe parallel Rust patterns.
- **Evolutionary Refinement:** We formalize both the data synthesis step and the final code generation process as an evolutionary search over the space of Abstract Syntax Trees (ASTs). Our agent generates a population of candidate solutions, compiles them, runs them against performance tests, and uses the error logs (or performance profiles) as “fitness functions” to drive mutation and crossover operations in the prompt space.
- **The Correctness-Speedup Trade-off:** We identify an “alignment tax” for concurrent programming. Our evaluation reveals that fine-tuning enables models to write significantly safer code at the expense of slightly slower peak performance (i.e. higher Pass@1 rates with lower Speedup@1 rates). This trade-off occurs because fine-tuned models learn to conservatively avoid raw, risky atomics in favor of stable, high-level primitives (such as `parlay::unique`).

Specifically, our paper succeeds in the following task:

¹<https://huggingface.co/qgggez/deepseek-parlay-6.7b>, <https://huggingface.co/qgggez/qwen3-30b-sft-stage2-merged>, https://huggingface.co/YangLiuWillow/qwen3_rust_dpo_final_merged

ParEVO democratizes parallel computing for irregular data by fine-tuning LLMs on verified primitives and deploying an evolutionary agent to iteratively optimize code based on runtime performance feedback.

All code can be found at <https://github.com/WildAlg/ParEVO> (ParEVO, 2026a).

Conflict of Interest Disclosure. D.A. and A.Y. are employed by Google DeepMind, which develops the Gemini family of models. This paper evaluates and uses Gemini models, including Gemini-2.5-Flash, Gemini-2.5-Pro, Gemini-3-Pro, and a fine-tuned Gemini-2.5-Pro variant referred to as Gemini-2.5-Parlay. This work was supported in part by a Google Academic Research Award.

2. Related Work

LLMs for Code Generation. Large Language Models have fundamentally shifted the landscape of software engineering, achieving remarkable success in sequential code completion (Husein et al., 2025), summarization (Ahmed & Devanbu, 2022), and translation (Eniser et al., 2024). Evaluation metrics have similarly evolved from surface n -gram overlap to structure-aware measures like CodeBLEU (Ren et al., 2020), which better correlate with functional correctness. However, current models struggle with complex planning and reasoning tasks (Kambhampati et al., 2024), a limitation that is magnified in High-Performance Computing (HPC). Nichols et al. (2024) demonstrated via the ParEval benchmark (Foundry et al., 2024) that while LLMs can generate syntactic structures for frameworks like Kokkos and MPI, they often fail to capture the semantic nuances of synchronization and race conditions. Recently, ParEval-Repo (Davis et al., 2025a;b) extended this evaluation to repository-level HPC translation tasks (e.g., multi-file codebases, build systems), highlighting that scaling beyond individual kernels introduces qualitatively different failure modes. Our work addresses this by moving beyond general-purpose pre-training, targeting the qualitatively harder regime of *parallel* and *irregular* algorithms where correctness requires respecting concurrency and performance depends on minimizing span.

Automated Parallelization and HPC Translation. Prior efforts in automated parallelization have largely focused on translating serial loops to OpenMP directives. BabelTower (Wen et al., 2022) previously tackled auto-parallelized program translation from sequential C to CUDA via a learning-based framework leveraging large-scale corpora and back-translation with reranking. OMPGPT (Chen et al., 2024a) fine-tunes GPT-Neo to predict pragmas for regular loops, while AutoParLLM (Mahmud et al., 2025) uses Graph Neural Networks to guide LLM generation based on parallelism patterns. TehraniJamsaz et al. (2024) attempts unsupervised

translation between languages and their HPC extensions but lacks a feedback mechanism for correctness. To address correctness risks such as subtle parallel bugs that often arise in serial-to-CUDA/OpenMP translation, MuSL (Ke et al., 2025; Ke, 2025; kcxain, 2025) proposed a mutual-supervision loop where a translator and a test-generator co-evolve: the tester synthesizes unit tests to filter translations, and the translator produces code to improve the tester. More recently, UniPar (Bitan et al., 2025) introduced a multi-agent framework for translating code between serial, OpenMP, and CUDA formats. While UniPar evaluates functional correctness (achieving 33%), it does not explicitly optimize for or benchmark the runtime scalability (work-span) of the generated algorithms. In contrast, ParEVO specifically targets *irregular* data, such as graph traversals and sparse matrix operations, where correct translation is insufficient, and *performance speedup* via parallelism, software engineering techniques, and performant algorithms and data structures is key. Recent advances have further specialized LLMs for parallel domains. For instance, Chaturvedi (2024) successfully fine-tuned base models on the HPC-Instruct dataset to target low-resource parallel languages, demonstrating that smaller, specialized models can match proprietary models on the ParEval benchmark. Similarly, frameworks like MARCO (Rahman, 2025) and PerfCoder (Yang, 2025) utilize multi-agent reasoning and execution trajectories to separate code generation from performance tuning. However, while these frameworks primarily target traditional imperative paradigms like OpenMP and CUDA, ParEVO specifically targets the algorithmic complexities of irregular data by grounding the model in the composable semantics of ParlayLib.

Structured Reasoning and Agentic Coding. To transcend the stochastic limitations of single-shot generation, frameworks like Reflexion (Shinn et al., 2023) use verbal reinforcement to iteratively correct failures. More recently, this paradigm has been extended via evolutionary search. Building upon this, EvoTune (Surina et al., 2025) augments LLM-based evolutionary program search by periodically updating the model via reinforcement learning on search-derived signals. Similarly, AI tree search systems (Aygün et al., 2025) embed LLM-based code mutation within a search procedure to maximize a measurable quality metric. To benchmark these search processes, AlgoTune (Press et al., 2025a;b) introduced a suite for numerical programs and evaluated an agent that iterates by editing, compiling, timing, and selecting the fastest valid variant. Concurrent open-source works such as OpenEvolve (Sharma, 2025a) have demonstrated the efficacy of coupling LLMs with genetic algorithms (Assumpção et al., 2025; Khrulkov et al., 2025; Novikov et al., 2025) and Quality-Diversity metrics (e.g., MAP-Elites) to prevent diversity collapse during program synthesis. ParEVO brings this evolutionary paradigm to the

HPC domain, replacing standard unit-test fitness functions with rigorous hardware profiling.

Abstractions for Irregular Parallelism. A core theme in parallel algorithmics is that abstraction choice determines accessibility. The classic work-span model (Brent, 1974) and work-stealing schedulers (Blumofe & Leiserson, 1999) provide a principled foundation for nested parallelism. High-level libraries like ParlayLib (Blelloch et al., 2020; Anderson et al., 2020) expose this theory through composable primitives (e.g., `scan`, `reduce`, `filter`), making provably efficient algorithms more accessible. Similarly, specialized abstractions such as GraphIt (Zhang et al., 2018; GraphIt-DSL et al., 2018) separate algorithm specification from scheduling choices to enable systematic performance tuning for irregular graph workloads, while Ligra (Shun & Blelloch, 2013) provides a lightweight shared-memory graph processing framework with simple vertex/edge mapping primitives and density-adaptive traversal strategies. Benchmarks like PBBS (Shun et al., 2012; Anderson et al., 2022) and Rusty-PBBS (Abdi et al., 2023a) formalize the evaluation of these irregular workloads. ParEVO leverages these insights by training models to target primitive-based code-writing within the Parlay ecosystem, ensuring that generated code is not just a parallel loop, but a structurally sound parallel algorithm capable of handling load imbalance inherent in irregular data (Sahu et al., 2019; Bronson et al., 2013).

Test-Time Compute and Execution Feedback. A growing consensus indicates that standard Supervised Fine-Tuning (SFT) and text-based reflection are insufficient for generating highly optimized code. Consequently, the field has rapidly shifted toward integrating real-machine execution feedback into the LLM reasoning loop. Using empirical hardware profiling as a direct reward signal drastically improves kernel efficiency (Du et al., 2025; Merouani et al., 2025; Lei et al., 2025). Crucially, Singh et al. (2024) applied test-time program search to the ParEval benchmark and empirically proved that LLMs exhibit a severe capability gap when attempting to act as their own “verifiers” for parallel code. This limitation directly motivates ParEVO’s Evolutionary Coding Agent (ECA), which sidesteps the unreliable “LLM-as-a-judge” paradigm in favor of treating deterministic compilers and sanitizers as ground-truth adversarial critics.

ML for Compiler/Performance Optimization. ML increasingly replaces heuristics for phase ordering, register allocation, auto-vectorization (Cummins et al., 2022; Trofin et al., 2021; Haj-Ali et al., 2020), and tensor scheduling (Chen et al., 2018; Zheng et al., 2020). Foundation models (Cummins et al., 2025; Tang et al., 2025) optimize IR using MCTS. For parallelization, OMPGPT, AutoParLLM, and UniPar predict pragmas for regular loops. However, they

operate at lower stack levels, lack robust feedback, or target regular matrices. ParEVO uniquely synthesizes high-level parallel abstractions for irregular data where vectorization fails.

Program Synthesis/Repair with Feedback. Agentic workflows increasingly use execution environments as oracles (Yang et al., 2023; Chen et al., 2024b; Ni et al., 2023; Zhou et al., 2024). While text-based critique and RL use compiler diagnostics, ParEVO advances this by tying selection pressure to scalable runtimes and problem-specific tests written by human experts that are designed to catch data races that lead to incorrect output and code that degrade performance.

3. Methodology: The ParEVO System

ParEVO is composed of three distinct stages: (1) Data Synthesis through Evolutionary Search, (2) Supervised Fine-Tuning, and (3) Inference-Time Evolutionary Search.

3.1. Stage 1: The Parlay-Instruct Fine-Tuning Dataset Corpus

The primary bottleneck for training “HPC-aware” LLMs is data scarcity. High-quality parallel C++ code is rare on GitHub compared to React components or Python scripts. We generated a synthetic dataset which incorporates parallel performance constructs, syntax, software engineering techniques, data structures, and algorithms, using a “Teacher-Student-Critic” pipeline via OpenEvolve (Sharma, 2025b). This synthetic dataset contains three parts: (1) the ParlayLib primitives, (2) DMOJ slow-fast code comparison pairs, and (3) DMOJ problem-solution pairs with labeled status, runtime performance, and any compiler or runtime error messages.

3.1.1. SEED GENERATION AND MUTATION

We manually authored 593 “golden” examples covering ParlayLib’s core primitives and 20 problems from DMOJ (DMOJ Developers, 2024). We then used Gemini-3-Pro (the “Teacher”) to mutate these seeds. We defined three mutation operators $\mathcal{M}$:

1. **Type Mutation** ($\mathcal{M}_{type}$): Changes the underlying data type (e.g., ‘int’ $\rightarrow$ ‘std::string’ or custom ‘struct Point’). This forces the model to learn C++ template instantiation rules.
2. **Constraint Mutation** ($\mathcal{M}_{cons}$): Adds logical predicates (e.g., “Sort only odd numbers” $\rightarrow$ requiring a ‘filter’ then ‘sort’). This forces the composition of primitives.
3. **Algorithmic Mutation** ($\mathcal{M}_{algo}$): Transforms the problem structure, e.g., converting a ‘reduce’ problem into

a ‘scan’ (prefix sum) problem.

We ran 5 passes of 10 mutations per seed, sampled from the operators $\{\mathcal{M}_{type}, \mathcal{M}_{cons}, \mathcal{M}_{algo}\}$ defined above, plus an additional 50 tasks targeting complex primitives (e.g., `parlay::delayed::filter`). This produced an initial pool of 29,700 candidates.

3.1.2. THE CRITIC LOOP: REJECTION SAMPLING

Let P be a generated problem and C be the generated code. We accept (P, C) into the dataset if and only if:

$$\text{Compile}(C) \wedge \text{UnitTest}(C) \quad (1)$$

That is, we only accept code that compiles and passes the unit tests. Compiling and executing each candidate against its unit test discarded 15,880 candidates that failed to compile or timed out. This filtration process yielded 13,820 verified instruction-tuning pairs, which we partitioned into a fine-tuning training set of size 13,120 and a held-out test set of 700 pairs for evaluation.

Performance Optimization Dataset. To enable the model to reason about runtime efficiency, we curated a benchmark of 20 challenging graph problems from the DMOJ competitive programming platform (DMOJ Developers, 2024). We synthesized optimization trajectories for these problems using the OpenEvolve framework (Sharma, 2025b) powered by Gemini-3-Pro. The data generation process followed the following novel protocol:

1. **Agent Initialization:** The agent was provided with the problem description and ParlayLib documentation, with a dual objective function minimizing both test failures and execution time.
2. **Trajectory Extraction:** We recorded the agent’s iterative refinements, extracting pairs of solutions (C_{base}, C_{opt}) from the evolutionary history.
3. **Speedup Threshold:** To ensure high-quality training signal, we filtered for pairs where the optimized solution C_{opt} achieved a runtime speedup of at least $1.2\times$ over C_{base} .

We constructed pairwise comparison examples using the solution pairs identified in the previous step. To eliminate positional bias, we randomized the assignment of “Code A” and “Code B” so that the faster implementation appears in either position with equal probability. The model is trained to identify the more performant solution using the following format:

Instruction: Determine which of the two code solutions has better performance.

Input:

Code A: [Source Code]

Code B: [Source Code]

Output: [Label of the Faster Solution]

A concrete example of this comparison format is provided in Figure 3.

While learning on performance edits has been used in (Shy-pula et al., 2024), our dataset is distinct in its focus on the complex, global transformations required for *irregular parallelism*, rather than the local sequential optimizations primarily targeted in the prior work.

Rust Parlay Primitives Given that the distinct Rust primitives were insufficient to constitute a robust fine-tuning dataset, we opted to include them directly in the context window. This approach allowed us to leverage the models’ pattern-matching and in-context learning capabilities without the need for parameter updates. To support this process, we integrated a full suite of Parlay-equivalent Rust primitives derived from **RPB** (Abdi et al., 2023b). Furthermore, to support the generation of higher-complexity algorithms, we manually implemented the delayed execution primitives in Rust and supplied them as immutable reference implementations within the system prompt.

Rust Evolutionary Dataset. To train the evolutionary coding agent for the Rust domain, we constructed a specialized dataset derived from the DMOJ benchmark execution logs. We aggregated the raw logs to extract code solutions, runtime metrics, and error traces. The data underwent a rigorous cleaning pipeline: we first filtered out irrelevant infrastructure failures (e.g., permission errors) and removed the held-out test set. We then deduplicated the remaining entries, prioritizing successful submissions while retaining a diverse set of failing attempts characterized by distinct error messages. The final corpus was serialized into JSONL format, where each entry explicitly pairs a problem description with the corresponding code, execution status, runtime performance, and any resulting compiler or runtime error messages. This rich metadata distinguishes our dataset from standard code corpora, enabling the model to learn both correct optimization patterns and specific error-correction strategies. Such a detailed corpus of training data is necessary for Rust given that Rust is notoriously difficult to use for irregular parallelism; hence, the available training data (including errors and compile-time messages) is rare for this language in the available base models.

3.2. Stage 2: Fine-Tuning DeepSeek, Gemini-2.5, Qwen3 for ParlayLib and Rust RPB

We selected **DeepSeek-6.7b-base** and **Qwen3-Coder-30B-A3B-Instruct** as our open-source backbones due to their

strong performance on standard C++. These models represent a tiered architecture strategy: DeepSeek-6.7b serves as our efficient, lightweight baseline, while Qwen3 acts as our high-capacity large model. We fine-tuned the model using Low-Rank Adaptation (LoRA) (Hu et al., 2022) to minimize compute costs while preserving the base model’s reasoning capabilities. We selected **Gemini-2.5-Pro** as our third base model due to its extensive context window for handling complex, long-context scenarios. All three models underwent fine-tuning to align them with our specific domain requirements.

Training Configuration. We configured the training pipeline according to model scale. For **DeepSeek-6.7b-base**, we executed single-stage Supervised Fine-Tuning (SFT) on an NVIDIA RTX 5000 Ada machine. We targeted the query and value projections using LoRA ($r = 8, \alpha = 16$) and trained on a combined dataset of ParlayLib syntax and ‘slow-fast’ performance pairs (FP16, learning rate $2e-4$).

For the larger **Qwen3-Coder-30B-A3B**, we implemented a dual-stage alignment pipeline on an NVIDIA H200 GPU. The first stage established domain capability via SFT on ParlayLib syntax and standard DMOJ solutions, using QLoRA ($r = 16, \alpha = 32$) across all linear attention and MLP layers. The second stage applied Direct Preference Optimization (DPO) to explicitly suppress failure modes. In this phase, we trained on contrastive triplets (pairing passing solutions against failing or inefficient implementations) using a reduced learning rate of $5e-6$ and $\beta = 0.1$.

Evaluation Environment. Performance benchmarks were conducted on a dual-socket compute node featuring two Intel Xeon Platinum 8562Y+ processors (64 physical cores total). To ensure consistent comparisons across frameworks, all experiments use 32 threads for OpenMP, ParlayLib, and Rust unless otherwise specified.

3.3. Stage 3: Evolutionary Coding Agent (ECA)

Evolutionary Search Strategy. To transcend the stochastic limitations of single-shot generation, we deploy an evolutionary agent that iteratively refines code for both correctness and performance. We model this process as a directed population-based search in the discrete space of possible programs. See Figure 1 for a diagram of the workflow.

The agent maintains a diverse population of candidate solutions, each associated with specific performance metrics (test coverage, execution time) and diagnostic artifacts (compiler logs, failure reasons, and targeted refinement instructions). The search initializes with either a baseline functional solution or a raw problem description. We define the

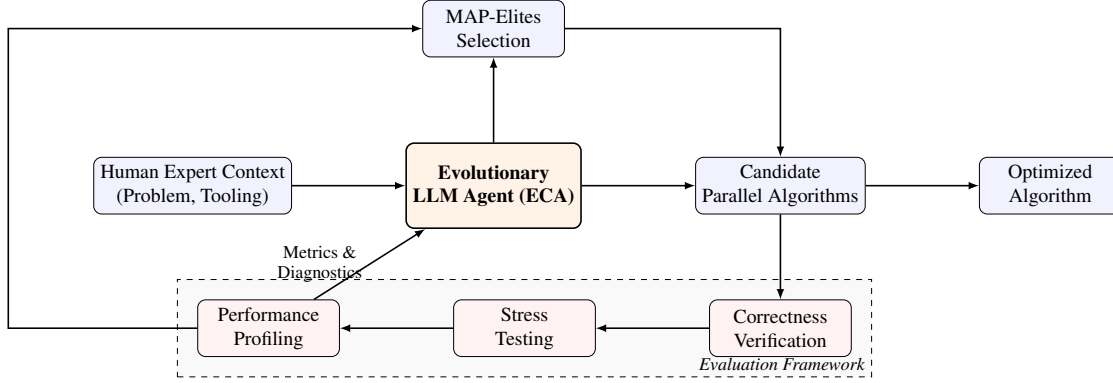


Figure 1. **Overview of the ParEVO Framework.** The system integrates human expert context (problem formulation, parallel tooling) with an evolutionary LLM agent. The cycle iteratively refines candidate parallel algorithms through a rigorous evaluation framework (correctness verification, stress testing, and performance profiling), using metrics to guide the selection of the next population via MAP-Elites.

fitness function $f(x)$ for a candidate solution x as:

$$f(x) = \begin{cases} 0 & \text{if } x \text{ fails compilation or tests} \\ \frac{1}{T(x)+\varepsilon} & \text{if } x \text{ passes, where } T(x) \text{ is runtime} \end{cases} \quad (2)$$

Furthermore, candidate solutions that trigger execution time-outs or yield inconsistent outputs across the five stress-test rounds are strictly assigned a fitness of 0.

A critical design choice in our evolutionary loop is reliance on deterministic external tools—specifically compilers and stress tests—rather than LLM-based static analysis. Because LLMs process code as text tokens, they natively fail to capture inter-thread timing and synchronization structures, making them prone to hallucinating data races. Furthermore, Singh et al. (2024) finds LLMs to be fundamentally unreliable verifiers of low-level parallel code. By executing each candidate five times on exceedingly large inputs (on the order of 10 million vertices), we force latent concurrency bugs to manifest as observable failures, providing a rigorous empirical filter that penalizes unsafe memory accesses.

In each generation, the agent selects survivors to populate the context window for the next iteration. To balance exploitation and exploration, we select the top $k = 3$ solutions by fitness (performance) and $d = 5$ diverse solutions via MAP-Elites. The MAP-Elites archive is organized as a grid indexed by two behavioral dimensions:

- **Complexity:** The character length of the source code, discretized into bins. Shorter programs fall into lower bins, helping retain solutions of varying structural complexity.
- **Diversity:** The mean character-level edit (Levenshtein) distance between a candidate and all other programs currently in the population. Programs structurally dissimilar to the rest of the population receive high diversity scores and occupy higher bins.

A new candidate replaces an incumbent in a grid cell only if its fitness is strictly higher, enforcing a strict quality-diversity invariant. These selected candidates, along with their diagnostic artifacts, prompt the LLM to synthesize the next generation of improved code. The process terminates by returning the candidate with the maximum fitness score.

3.4. Supported Languages

To demonstrate the versatility of our approach, in this paper, we use our ParEVO on two languages: C++ and Rust. For C++, we use our ParEVO system to fine-tune models on ParlayLib (Blelloch et al., 2020). For Rust, we use our ParEVO system to fine-tune models on RPB: Rust Parallel Benchmarks Suite (Abdi et al., 2023a;b). For both C++ and Rust, our methods lead to improved performance.

3.5. Benchmarking Suite

We evaluate our framework across four distinct benchmarks to assess both generation quality and runtime performance. First, we compare our fine-tuned models against state-of-the-art local and commercial LLMs using the **ParEval** (Nichols et al., 2024) library. Second, we measure absolute performance against expert human baselines, utilizing C++ solutions from **PBBSBench** (Shun et al., 2012) and Rust implementations from **RPB** (Abdi et al., 2023b). Finally, to test generalization, we evaluate on a held-out set of **DMOJ** competitive programming problems. In this setting, we compare the runtime of code generated by ParEVO against official contest solutions, demonstrating significant speedups.

4. Experimental Results

4.1. Experimental Setup

Hardware. All experiments were conducted on a dual-socket compute node equipped with two Intel Xeon Plat-

inum 8562Y+ processors (64 physical cores total) and 512GB DDR5 ECC RAM. An NVIDIA H200 GPU was utilized solely for inference.

Benchmarks. We evaluated on:

1. **ParEval:** The testing suite of (Nichols et al., 2024).
2. **PBBSBench & RPB:** Expert-written C++ and Rust baselines (Shun et al., 2012; Abdi et al., 2023b).
3. **DMOJ:** A held-out set of competitive programming problems. (DMOJ Developers, 2024)

4.2. Main Results: ParEval Performance

Methodological Note on Expected Speedup. In traditional systems literature, the geometric mean is typically used to average normalized execution times of a static benchmark suite across different hardware. However, in the context of zero-shot code generation over a large distribution of tasks (ParEval), we conceptualize performance formally as an expected capability reward. Specifically, we report the arithmetic mean of Speedup@1 to represent the *expected speedup* ($\mathbb{E}[S]$) a user would experience when querying the model with a random task from the problem domain. This aligns directly with standard machine learning evaluation practices for reporting expected test-time rewards over a distribution, as opposed to summarizing the total execution time of a fixed static workload.

Table 1 presents the performance of local and commercial models. Our fine-tuned models (**Gemini-2.5-Parlay** and **DeepSeek-Parlay**) significantly outperform their base counterparts. Notably, **Gemini-2.5-Parlay** achieves an average $107\times$ speedup over the baseline provided by ParEval, driven by its ability to generate valid, compilable parallel code (Build@1 0.81 vs 0.25 of the state-of-the-art Gemini 3.0 Pro). Even our smallest fine-tuned model, DeepSeek-Parlay (with 6.7b parameters) is able to beat the commercial state-of-the-art Gemini-3-Pro. The Speedup@1 metric is the arithmetic mean of the expected best performance speedups across all 59 problems relative to a sequential baseline. The exceptionally high $107.43\times$ mean for Gemini-2.5-Parlay is driven by heavy-tailed performance on specific irregular tasks (e.g., `34_scan_largest_contiguous_subarray_sum`), where the model discovered an optimal prefix-sum algorithm while the baseline sequential implementation is heavily bottlenecked by nested loops.

Impact of Fine-tuning on Code Quality. As illustrated in Figure 5, the most consistent effect of fine-tuning is on runtime performance: ParEVO produces faster code in eleven of twelve categories, frequently by an order of magnitude, indicating that the model learns to select more efficient

parallel patterns by using ParlayLib primitives rather than merely valid ones. The effect on Build@1 and Pass@1 is category-dependent. Fine-tuning yields its largest correctness gains precisely where the base model was weakest at expressing the appropriate ParlayLib idiom—most notably graph (Build@1 : 0.62 $\rightarrow$ 0.97, Pass@1 : 0.42 $\rightarrow$ 0.76) and histogram (Pass@1 : 0.19 $\rightarrow$ 0.63), while incurring mild regressions on categories the base already handled well (e.g., `fft`, `geometry`, `search`).

4.3. Semantic Alignment via Fine-Tuning

A critical advantage of ParEVO is its ability to learn the correct semantics of parallel primitives. In the complex number sorting task (Figure 21 in Appendix), the base model failed completely ($\text{Build@1} = 0$), struggling with C++ custom comparators. The fine-tuned model not only compiled ($\text{Build@1} = 1$) but achieved a speedup of $17.5\times$. This suggests that the model has learned to navigate the complex type system of ParlayLib.

4.4. Performance Analysis: Strong Scaling

Code correctness is insufficient for HPC; the solution must also scale. Figure 2 demonstrates strong scaling up to 64 cores. For regular parallelism like Discrete Fourier Transform, our model generates code that scales near-linearly ($40\times$ speedup), abstracting away complex synchronization that typically hinders manual implementations. The performance drop observed at 64 cores for Figure 2(d) is due to parallel overhead and thread contention, where the generated code reveals that the LLM attempted to maintain local dynamic queues inside a nested parallel BFS loop.

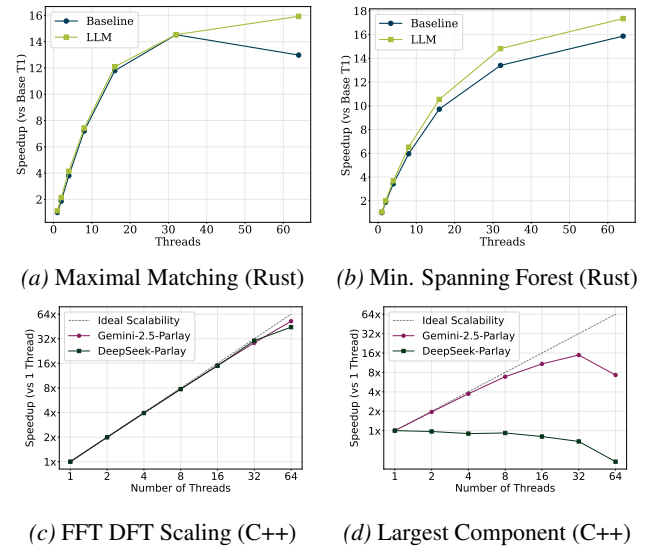


Figure 2. Strong Scaling results. (c) Algorithms like Discrete Fourier Transform show excellent scaling with ParEVO’s generated code, reaching nearly $40\times$ speedup on 64 cores.

Execution Model	Code	Sched.	Build@1	Pass@1	Speedup@1 (AM)	Speedup@1 (GM)
Claude Opus 4.5	Parlay	Parlay	0.97	0.05	0.42	1.13
GPT-5 Thinking	Parlay	Parlay	0.93	0.05	0.43	1.13
Gemini-2.5-Flash	Parlay	Parlay	0.57	0.29	6.88	2.75
Gemini-2.5-Pro	Parlay	Parlay	0.84	0.54	85.34	5.47
Gemini-3-Pro	Parlay	Parlay	0.25	0.23	3.68	1.53
Gemini-2.5-Parlay	Parlay	Parlay	0.81	0.58	107.43	6.33
DeepSeek-6.7B-Base	Parlay	Parlay	0.89	0.11	0.73	1.14
DeepSeek-Syntax	Parlay	Parlay	0.85	0.12	2.42	1.38
DeepSeek-Parlay	Parlay	Parlay	0.81	0.26	126.16	3.14
Qwen3-Parlay	Parlay	Parlay	0.50	0.33	5.01	1.87
DeepSeek-Coder-V2-Lite-Base	Parlay	Parlay	0.80	0.09	1.30	1.19
Qwen2.5-Coder-32B	Parlay	Parlay	0.93	0.11	5.02	1.82
Qwen2.5-Coder-32B-Instruct	Parlay	Parlay	0.61	0.41	7.00	2.22
DeepSeek-Coder-V2-Lite-Base	Rust	Rayon	0.73	0.29	3.47	0.58
DeepSeek-Coder-V2-Lite-Instruct	Rust	Rayon	0.40	0.02	0.15	1.04
Qwen2.5-Coder-32B	Rust	Rayon	0.82	0.45	4.43	0.63
Qwen2.5-Coder-32B-Instruct	Rust	Rayon	0.63	0.49	4.61	0.57
Qwen3-Coder-30B-Instruct	Rust	Rayon	0.61	0.50	4.57	0.46
Qwen3-Rust	Rust	Rayon	0.64	0.46	4.62	0.35
StarCoder2-15B	Rust	Rayon	0.77	0.27	2.46	0.79
Gemini-3-Pro	Rust	Rayon	0.97	0.82	7.32	0.69

Table 1. ParEval results (temperature = 0.2). **Code** denotes the parallel programming language used, and **Scheduler** the parallel runtime. *Parlay* code uses the ParlayLib library, with either ParlayLib’s internal scheduler or OpenMP as the scheduling backend. *Rust* code uses Rayon’s dynamic work-stealing scheduler. Our fine-tuned models achieve orders-of-magnitude improvements in speedup.

4.5. Comparison vs. Expert Baselines

We benchmarked our generated solutions against expert human implementations from PBBSBench (C++) and RPB (Rust). As shown in Table 2, ParEVO matches or exceeds expert performance. For **Maximal Independent Set**, the generated Rust solution achieved a $4.1\times$ speedup over the baseline by identifying a superior parallel strategy. We demonstrate the maximum speedups we can gain by using Gemini-3-Pro with our ParEVO evolutionary strategy described in Section 3.3.

4.6. Ablation Study: Evolutionary Agent

To isolate the contribution of the Evolutionary Coding Agent (ECA), we evaluated performance with the agent disabled. Table 3 confirms that iterative refinement is crucial: 30 iterations of ECA yield a $2.2\times$ performance multiplier over single-shot generation.

For this ablation study, we utilized a reserved set of training problems sourced from DMOJ.² In Table 3, the base performance ($1.00\times$) corresponds to `baseline.cpp`, which we define as the very first solution that passes all tests (this is not necessarily the solution from iteration 1 if the early

²The DMOJ training problems are available at <https://github.com/WildAlg/ParEVO/tree/main/code-contests-dataset>

attempts fail). The reported speedup for the ECA configurations is calculated as the average of the relative speedups achieved across all datasets within this training corpus.

While the actual prompt in each iteration contains more context (other iterations/metrics) by the default template of *openevolve*, the structural system prompt we specified for the ECA is provided in Appendix A.

4.7. Analysis: The Correctness-Speedup Trade-off

A deeper analysis of Graph problems (Table 4) reveals a trade-off. Fine-tuning increases correctness (Pass@1 $0.42 \rightarrow 0.76$) by enforcing safe API usage, but sometimes degrades speedup ($21\times \rightarrow 13\times$) as it favors stable, high-level primitives over risky, fine-grained atomic operations.

5. Discussion and Limitations

5.1. The Role of Abstraction in Parallelization

Our findings suggest that the efficacy of LLM parallel code generation depends heavily on the abstraction level provided by the target intermediate representation (IR). We argue that the superior performance of ParEVO on ParlayLib stems from an *alignment of abstraction*. Imperative models like OpenMP force the LLM to manage global state and explicit synchronization: tasks that maximize the “state-tracking”

Table 2. Runtime Comparison: PBBS & RPB at Thread=32, best speedup across test inputs. Baseline code is state-of-the-art human-written code. We also demonstrate the speedup against one thread, labeled Speedup (1T).

PROBLEM	MODEL/METHOD	LANGUAGE	RUNTIME (S)	SPEEDUP (1T)	SPEEDUP (BASE)
MAXIMAL INDEPENDENT SET	BASILINE	RUST	0.31876	1.116×	—
MAXIMAL INDEPENDENT SET	PAVEVO (GEMINI)	RUST	0.07728	0.938×	4.125×
MAXIMAL MATCHING	BASILINE	RUST	0.20646	21.723×	—
MAXIMAL MATCHING	PAVEVO (GEMINI)	RUST	0.1928	21.43286835×	1.0708×
MINIMUM SPANNING FOREST	BASILINE	RUST	0.41968	13.427×	—
MINIMUM SPANNING FOREST	PAVEVO (GEMINI)	RUST	0.38004	13.87×	1.1043×
SPANNING FOREST	BASILINE	RUST	0.11571	10.378×	—
SPANNING FOREST	PAVEVO (GEMINI)	RUST	0.08865	15.482×	1.3052×
MINIMUM SPANNING FOREST	BASILINE	C++	1.24	22.633×	—
MINIMUM SPANNING FOREST	PAVEVO (GEMINI)	C++	1.169	23.689×	1.061×
HISTOGRAM	BASILINE	C++	0.051	27.59×	—
HISTOGRAM	PAVEVO (GEMINI)	C++	0.019	> 13.94×	2.68421×

Table 3. Impact of Evolutionary Refinement on Speedup. 30 iterations of ECA yield a 2.2× speedup over the first valid solution.

CONFIGURATION	SPEEDUP
GEMINI-3-PRO (NO ECA)	1.00× (BASELINE)
GEMINI-3-PRO + ECA (10 ITER)	1.498×
GEMINI-3-PRO + ECA (30 ITER)	2.218×

Table 4. Performance on Graph Problems. Fine-tuning improves reliability (Pass@1) but favors safer, slightly slower algorithms.

Model	Build@1	Pass@1	Speedup@1
Gemini-2.5-Pro	0.62	0.42	21.76
Gemini-2.5-Parlay	0.97	0.76	13.67

burden on the attention mechanism and increase the probability of race conditions.

In contrast, ParlayLib functions as a high-level parallel DSL. Its functional primitives (e.g., `map`, `reduce`, `scan`) encapsulate complex scheduling logic and enforce immutability. This reduces parallelization to *local* transformations (mapping serial loops to equivalent functional constructs) which aligns naturally with the token-local prediction of Transformer models.

By training our models to target ParlayLib’s composable primitives, ParEVO aligns the optimization objective with the token-local reasoning capabilities of the Transformer architecture, yielding code that is both mathematically sound and highly performant.

5.2. Limitations and Future Directions

- **Architectural Scope:** ParEVO is currently optimized for shared-memory multicore architectures. It does not address the distributed memory paradigm (e.g. MPI/P-GAS), where communication latency and data partitioning introduce distinct optimization constraints.

- **Inference Latency vs. Runtime Efficiency:** The Evolutionary Coding Agent trades inference-time compute for execution-time speedup. While generating multiple candidates and compiling them is costly, we argue this is an acceptable amortized cost for HPC kernels that may run trillions of times over their lifecycle.
- **Domain Generalization:** On some benchmarks, the model produces “confident hallucinations” when applying learned parallel patterns to unfamiliar algorithmic domains. Future work will explore integrating formal verification tools into the evolutionary loop to constrain these semantic errors.

6. Conclusion

We have presented **ParEVO**, a framework that bridging modern generative AI and high-performance computing. By curating a specialized dataset of parallel primitives and fine-tuning models to internalize the **Work-Depth** cost model, we achieve state-of-the-art results on the ParEval, surpassing both commercial LLMs and traditional heuristics.

Crucially, our results reveal that syntax generation alone is insufficient for HPC. The integration of an **Evolutionary Coding Agent**—which treats the compiler and runtime profiler as adversarial critics—is essential for traversing the optimization landscape. This work sets a precedent for **AI-Driven Performance Engineering**: moving beyond code completion to systems that actively reason about scalability, correctness, and the complex interplay between algorithms and hardware.

Acknowledgements

We thank Lin Zhong for helpful discussions and Rust resources, Ramla Ijaz for helpful discussions, and Roger Fu for compiling and providing to us the publicly available test cases for the competitive programming problems we used.

We also thank the extended team at Google DeepMind who supported this research direction. Amir Yazdanbakhsh and Deniz Altınbüken contributed to this paper in an advisory capacity.

This work was supported in part by the National Science Foundation (NSF) under Grant #CCF-2453323 and a Google Academic Research Award.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning and High Performance Computing. By enabling easier access to efficient parallel programming, this work could reduce the energy footprint of large-scale computations as well as make parallel computing accessible to non-experts. However, as with all code generation tools, there is a risk of generating subtle bugs in critical systems if not properly verified. We recommend human oversight for mission-critical applications.

References

- Abdi, J., Zhang, G., and Jeffrey, M. C. Brief announcement: Is the problem-based benchmark suite fearless with rust? In *Proceedings of the 35th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA '23)*. ACM, 2023a. doi: 10.1145/3558481.3591313.
- Abdi, J., Zhang, G., and Jeffrey, M. C. Rusty-PBBS: Rust problem based benchmark suite. <https://github.com/mcj-group/rpb>, 2023b. GitHub repository.
- Ahmed, T. and Devanbu, P. Learning code summarization from a small and local dataset, 2022. URL <https://arxiv.org/abs/2206.00804>.
- Anderson, D., Bletloch, G., Dhulipala, L., Tseng, T., wheatman, Hübschle, L., Yesantharao, R., Dong, X., and aheydon google. Parlaylib: A toolkit for programming parallel algorithms on shared-memory multicore machines (github repository; cmu parlay group), 2020. URL <https://github.com/cmuparlay/parlaylib>.
- Anderson, D., Bletloch, G. E., Dhulipala, L., Dobson, M., and Sun, Y. The problem-based benchmark suite (PBBS), V2. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '22)*. ACM, 2022. doi: 10.1145/3503221.3508422.
- Assumpção, H., Ferreira, D., Campos, L., and Murai, F. Codeevolve: an open source evolutionary coding agent for algorithm discovery and optimization. *arXiv preprint arXiv:2510.14150*, 2025.
- Aygün, E., Belyaeva, A., Comanici, G., Coram, M., Cui, H., Garrison, J., Johnston, R., Kast, A., McLean, C. Y., Norgaard, P., Shamsi, Z., Smalling, D., Thompson, J., Venugopalan, S., Williams, B. P., He, C., Martinson, S., Plomecka, M., Wei, L., Zhou, Y., Zhu, Q.-Z., Abraham, M., Brand, E., Bulanova, A., Cardille, J. A., Co, C., Ellsworth, S., Joseph, G., Kane, M., Krueger, R., Kartiwa, J., Liebling, D., Lueckmann, J.-M., Raccuglia, P., Wang, X. J., Chou, K., Manyika, J., Matias, Y., Platt, J. C., Dorfman, L., Mourad, S., and Brenner, M. P. An ai system to help scientists write expert-level empirical software. *arXiv preprint arXiv:2509.06503*, 2025. doi: 10.48550/ARXIV.2509.06503. URL <https://arxiv.org/abs/2509.06503>.
- Bitan, T., Kadosh, T., Kaplan, E., Meiri, S., Chen, L., Morales, P., Hasabnis, N., and Oren, G. Unipar: A unified llm-based framework for parallel and accelerated code translation in hpc. In *2025 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2025.
- Bletloch, G. E., Anderson, D., and Dhulipala, L. ParlayLib: A Toolkit for Parallel Algorithms on Shared-Memory Multicore Machines. In *Proceedings of the 32nd ACM Symposium on Parallelism in Algorithms and Architectures, SPAA '20*, pp. 507–509, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450369350. doi: 10.1145/3350755.3400254. URL <https://doi.org/10.1145/3350755.3400254>.
- Blumofe, R. D. and Leiserson, C. E. Scheduling multi-threaded computations by work stealing. *J. ACM*, 46 (5):720–748, September 1999. ISSN 0004-5411. doi: 10.1145/324133.324234. URL <https://doi.org/10.1145/324133.324234>.
- Brent, R. P. The parallel evaluation of general arithmetic expressions. *Journal of the ACM*, 21(2):201–206, 1974.
- Bronson, N., Amsden, Z., Cabrera, G., Chakka, P., Dimov, P., Ding, H., Ferris, J., Giardullo, A., Kulkarni, S., Li, H., Marchukov, M., Petrov, D., Puzar, L., Song, Y. J., and Venkataramani, V. Tao: Facebook’s distributed data store for the social graph. In *Proceedings of the 2013 USENIX Conference on Annual Technical Conference, USENIX ATC'13*, pp. 49–60, USA, 2013. USENIX Association.
- Chaturvedi, A. Hpcoder-v2: Efficient fine-tuning of small language models for high-performance computing. *arXiv preprint arXiv:2410.20527*, 2024.
- Chen, L., Bhattacharjee, A., Ahmed, N., Hasabnis, N., Oren, G., Vo, V., and Jannesari, A. OMPGPT: A Generative Pre-trained Transformer Model for OpenMP, pp. 121–134. Springer Nature Switzerland, 2024a. ISBN 9783031695773. doi: 10.1007/

- 978-3-031-69577-3_9. URL http://dx.doi.org/10.1007/978-3-031-69577-3_9.
- Chen, T., Zheng, L., Yan, E., Jiang, Z., Moreau, T., Ceze, L., Guestrin, C., and Krishnamurthy, A. Learning to optimize tensor programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 3393–3404, 2018.
- Chen, X., Lin, M., Schärli, N., and Zhou, D. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024b.
- Cummins, C., Wasti, B., Guo, J., Cui, B., Ansel, J., Gomez, S., Jain, S., Liu, J., Teytaud, O., Steiner, B., Tian, Y., and Leather, H. CompilerGym: Robust, performant compiler optimization environments for AI research. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pp. 92–105, 2022. doi: 10.1109/CGO53902.2022.9741258.
- Cummins, C., Seeker, V., Grubisic, D., Rozière, B., Gehring, J., Synnaeve, G., and Leather, H. LLM compiler: Foundation language models for compiler optimization. In *Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction (CC)*, 2025. doi: 10.1145/3708493.3712691.
- Davis, J. H., Nichols, D., Khillan, I., and Bhatele, A. Pareval-repo: A benchmark suite for evaluating llms with repository-level hpc translation tasks. In *Proceedings of the International Conference on Parallel Processing (ICPP 2025)*, 2025a. doi: 10.1145/3754598.3754669. URL <https://doi.org/10.1145/3754598.3754669>.
- Davis, J. H., Nichols, D., Khillan, I., and Bhatele, A. Pareval-repo: A benchmark suite for evaluating llms with repository-level hpc translation tasks. *arXiv preprint arXiv:2506.20938*, 2025b. doi: 10.48550/ARXIV.2506.20938. URL <https://arxiv.org/abs/2506.20938>.
- DMOJ Developers. DMOJ: Modern online judge. <https://github.com/DMOJ/online-judge>, 2024. GitHub repository.
- Du, M., Tuan, L. A., Liu, Y., Qing, Y., Huang, D., He, X., Liu, Q., Ma, Z., and kiong Ng, S. Afterburner: Reinforcement learning facilitates self-improving code efficiency optimization. *arXiv preprint arXiv:2505.23387*, 2025.
- Eniser, H. F., Zhang, H., David, C., Wang, M., Christakis, M., Paulsen, B., Dodds, J., and Kroening, D. Towards translating real-world code with llms: A study of translating to rust, 2024. URL <https://arxiv.org/abs/2405.11514>.
- Foundry, P. C., Nichols, D., Zi, Y., Xie, Z., and Menon, H. Pareval: Parallel code evaluation benchmark (github repository), 2024. URL <https://github.com/parallelcodefoundry/ParEval>.
- GraphIt-DSL, Zhang, Y., Manlaibaatar, T., Brahmakshatriya, A., ykenny1, Baghdadi, R., Furst, E., Siegfried, G., Wade, B., and Dhulipala, L. Graphit-dsl/graphit: Graphit compiler and dsl implementation (github repository), 2018. URL <https://github.com/GraphIt-DSL/graphit>.
- Haj-Ali, A., Ahmed, N. K., Willke, T. L., Shao, Y. S., Asanovic, K., and Stoica, I. NeuroVectorizer: End-to-end vectorization with deep reinforcement learning. In *Proceedings of the 18th ACM/IEEE International Symposium on Code Generation and Optimization (CGO)*, pp. 242–255. ACM, 2020. doi: 10.1145/3368826.3377928.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2022. URL <https://openreview.net/pdf?id=nZeVKeeFYf9>.
- Husein, R. A., Aburajouh, H., and Catal, C. Large language models for code completion: A systematic literature review. *Computer Standards & Interfaces*, 92:103917, 2025. ISSN 0920-5489. doi: <https://doi.org/10.1016/j.csi.2024.103917>. URL <https://www.sciencedirect.com/science/article/pii/S0920548924000862>.
- Kambhampati, S., Valmeekam, K., Guan, L., Verma, M., Stechly, K., Bhambri, S., Saldyt, L., and Murthy, A. Llm’s can’t plan, but can help planning in llm-modulo frameworks, 2024. URL <https://arxiv.org/abs/2402.01817>.
- kcxain. kcxain/translator-qwen3-0.6b: Musl c-to-cuda translator model (hugging face), 2025. URL <https://huggingface.co/kcxain/translator-Qwen3-0.6B>.
- Ke, C. kcxain/musl: code repository for mutual-supervised learning for sequential-to-parallel code translation, 2025. URL <https://github.com/kcxain/musl>.
- Ke, C., Zhang, R., Wang, S., Ding, L., Li, G., Wen, Y., Zhang, S., Xu, R., Qin, J., Guo, J., Wang, C., Li, L., Guo, Q., and Chen, Y. Mutual-supervised learning for sequential-to-parallel code translation. *arXiv preprint arXiv:2506.11153*, 2025. doi: 10.48550/ARXIV.2506.11153. URL <https://arxiv.org/abs/2506.11153>.

- Khrulkov, V., Galichin, A., Bashkirov, D., Vinichenko, D., Travkin, O., Alferov, R., Kuznetsov, A., and Oseledets, I. Gigaevo: An open source optimization framework powered by llms and evolution algorithms. *arXiv preprint arXiv:2511.17592*, 2025.
- Lei, K., Yang, H., Zhang, H., You, X., Zhang, K., Luan, Z., Liu, Y., and Qian, D. Pragma: A profiling-reasoned multi-agent framework for automatic kernel optimization. *arXiv preprint arXiv:2511.06345*, 2025.
- Mahmud, Q. I., TehraniJamsaz, A., Phan, H. D., Chen, L., Capotă, M., Willke, T. L., Ahmed, N. K., and Jannesari, A. AutoParLLM: GNN-guided context generation for zero-shot code parallelization using LLMs. In *Chiruzzo, L., Ritter, A., and Wang, L. (eds.), Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 11821–11841, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.593. URL <https://aclanthology.org/2025.naacl-long.593/>.
- Merouani, M., Bernou, I. K., and Baghdadi, R. Agentic auto-scheduling: An experimental study of llm-guided loop optimization. *arXiv preprint arXiv:2511.00592*, 2025.
- Ni, A., Iyer, S., Radev, D., Stoyanov, V., Yih, W.-t., Wang, S. I., and Lin, X. V. LEVER: Learning to verify language-to-code generation with execution. In *International Conference on Machine Learning (ICML)*, 2023.
- Nichols, D., Davis, J. H., Xie, Z., Rajaram, A., and Bhatele, A. Can large language models write parallel code? In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing (HPDC '24)*. ACM, 2024. doi: 10.1145/3625549.3658689. URL <https://pssg.cs.umd.edu/assets/papers/2024-06-pareval-hpdc.pdf>.
- Novikov, A., Vü, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J. R., Mehrabian, A., Kumar, M. P., See, A., Chaudhuri, S., Holland, G., Davies, A., Nowozin, S., Kohli, P., and Balog, M. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- ParEVO. Parevo project repository (code/data for parevo; includes parlay-instruct artifacts), 2026a. URL <https://github.com/WildAlg/ParEVO>.
- ParEVO. Deepseek-parlay-6.7b: A fine-tuned model for parallel algorithmic reasoning, 2026b. URL <https://huggingface.co/qgggez/deepseek-parlay-6.7b>.
- ParEVO. Qwen3-30b-sft-stage2-merged, 2026. URL <https://huggingface.co/qgggez/qwen3-30b-sft-stage2-merged>.
- ParEVO. Qwen3-rust-dpo: A fine-tuned model for safe parallel rust, 2026. URL https://huggingface.co/YangLiuWillow/qwen3_rust_dpo_final_merged.
- Press, O., Amos, B., Zhao, H., Wu, Y., Ainsworth, S. K., Krupke, D., Kidger, P., Sajed, T., Stellato, B., Park, J., Bosch, N., Meril, E., Steppi, A., Zharmagambetov, A., Zhang, F., Perez-Pineiro, D., Mercurio, A., Zhan, N., Abramovich, T., Lieret, K., Zhang, H., Huang, S., Bethge, M., and Press, O. Algotune: Can language models speed up general-purpose numerical programs? *arXiv preprint arXiv:2507.15887*, 2025a. doi: 10.48550/ARXIV.2507.15887. URL <https://arxiv.org/abs/2507.15887>.
- Press, O., Amos, B., Zhao, H., Wu, Y., Ainsworth, S. K., Krupke, D., Kidger, P., Sajed, T., Stellato, B., Park, J., Bosch, N., Meril, E., Steppi, A., Zharmagambetov, A., Zhang, F., Perez-Pineiro, D., Mercurio, A., Zhan, N., Abramovich, T., Lieret, K., Zhang, H., Huang, S., Bethge, M., and Press, O. Algotune benchmark dataset, 2025b. URL <https://huggingface.co/datasets/oripress/AlgoTune>.
- Rahman, M. Marco: Multi-agent reasoning for code optimization. *arXiv preprint arXiv:2501.12345*, 2025.
- Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., Sundaresan, N., Zhou, M., Blanco, A., and Ma, S. Codebleu: a method for automatic evaluation of code synthesis, 2020. URL <https://arxiv.org/abs/2009.10297>.
- Sahu, S., Mhedhbi, A., Salihoglu, S., Lin, J., and Özsu, M. T. The ubiquity of large graphs and surprising challenges of graph processing: extended survey. *The VLDB Journal*, 29(2–3):595–618, June 2019. ISSN 0949-877X. doi: 10.1007/s00778-019-00548-x. URL <http://dx.doi.org/10.1007/s00778-019-00548-x>.
- Sharma, A. Openevolve: an open-source evolutionary coding agent, 2025a. URL <https://github.com/algorithmicsuperintelligence/openevolve>.
- Sharma, A. Openevolve: An open source implementation of google deepmind’s alphaevolve. <https://github.com/codelion/openevolve>, 2025b. Accessed: 2026-01-24.
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

- Shun, J. and Blleloch, G. E. Lagra: a lightweight graph processing framework for shared memory. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pp. 135–146. ACM, 2013. doi: 10.1145/2442516.2442530. URL <https://doi.org/10.1145/2442516.2442530>.
- Shun, J., Blleloch, G. E., Kyrola, A., Simhadri, H. V., Tangwongsan, K., Fineman, J. T., and Gibbons, P. B. Brief announcement: The problem based benchmark suite. In *Proceedings of the 24th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA ’12)*. ACM, 2012. doi: 10.1145/2312005.2312018.
- Shypula, A., Madaan, A., Zeng, Y., Alon, U., Gardner, J., Hashemi, M., Neubig, G., Ranganathan, P., Bastani, O., and Yazdanbakhsh, A. Learning performance-improving code edits, 2024. URL <https://arxiv.org/abs/2302.07867>.
- Singh, G., Guha, A., Kailkhura, B., and Menon, H. Can test-time compute help LLMs write low-resource parallel code better? In *NeurIPS Workshop on Deep Learning for Code (DLAC)*, 2024. URL <https://openreview.net/forum?id=0RnJzt8v84>.
- Surina, A., Mansouri, A., Quaedyvlieg, L., Seddas, A., Vizovska, M., Abbe, E., and Gulcehre, C. Algorithm discovery with llms: Evolutionary search meets reinforcement learning. *CoRR*, abs/2504.05108, 2025. doi: 10.48550/ARXIV.2504.05108. URL <https://doi.org/10.48550/arXiv.2504.05108>.
- Tang, A. S., Priebe, C., Mahapatra, R., Qin, L., and Esmaeilzadeh, H. Reasoning compiler: LLM-guided optimizations for efficient model serving. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- TehraniJamsaz, A., Bhattacharjee, A., Chen, L., Ahmed, N. K., Yazdanbakhsh, A., and Jannesari, A. CoderoSetta: pushing the boundaries of unsupervised code translation for parallel programming. In *Proceedings of the 38th International Conference on Neural Information Processing Systems, NIPS ’24*, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9798331314385.
- Trofin, M., Qian, Y., Brevdo, E., Lin, Z., Choromanski, K., and Li, D. MLGO: A machine learning guided compiler optimizations framework. *arXiv preprint arXiv:2101.04808*, 2021.
- Wen, Y., Guo, Q., Fu, Q., Li, X., Xu, J., Tang, Y., Zhao, Y., Hu, X., Du, Z., Li, L., Wang, C., Zhou, X., and Chen, Y. Babeltower: Learning to auto-parallelized program translation. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 23685–23700. PMLR, 2022. doi: unspecified. URL <https://proceedings.mlr.press/v162/wen22b.html>.
- Yang, J., Prabhakar, A., Narasimhan, K., and Yao, S. Intercode: standardizing and benchmarking interactive coding with execution feedback. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS ’23*, Red Hook, NY, USA, 2023. Curran Associates Inc.
- Yang, Z. Perfcoder: Performance-driven code generation. *arXiv preprint arXiv:2502.54321*, 2025.
- Zhang, Y., Yang, M., Baghdadi, R., Kamil, S., Shun, J., and Amarasinghe, S. P. Graphit: a high-performance graph dsl. *Proceedings of the ACM on Programming Languages*, 2(OOPSLA):121:1–121:30, 2018. doi: 10.1145/3276491. URL <https://doi.org/10.1145/3276491>.
- Zheng, L., Jia, C., Sun, M., Wu, Z., Yu, C. H., Haj-Ali, A., Wang, Y., Yang, J., Zhuo, D., Sen, K., Gonzalez, J. E., and Stoica, I. Ansor: Generating high-performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pp. 863–879. USENIX Association, 2020.
- Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., and Wang, Y.-X. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pp. 62138–62160. PMLR, 2024.

A. Evolutionary Coding Agent (ECA) System Prompt

The structural system prompt we specify for the single ECA node is as follows:

ECA System Prompt

```
You are an expert C++ competitive programmer. Your task is to write a
COMPLETE, CORRECT, and FAST C++ solution.

PROBLEM:
{problem_description}

REQUIREMENTS:
Write a complete C++ parallel program that compiles and runs correctly
Read input from standard input (cin)
Write output to standard output (cout)
Handle all edge cases mentioned in the problem
Optimize for speed - use efficient algorithms and data structures
Use C++ STL where appropriate (vector, map, set, priority_queue, etc.)
Consider time complexity and space complexity
The parlay library MUST be used as the core computation of the program

AVAILABLE LIBRARIES:
Standard C++ libraries (iostream, algorithm, vector, map, etc.)
The parlay library

Note:
parlay::parallel_for does not guarantee ordering, do not use it with IO operations.

CODE STYLE:
Use C++ style comments: // for single line, /* */ for multi-line
Do NOT use Python-style # comments
Comments should be simple and short
Include necessary headers
Write clean, readable code

OUTPUT FORMAT:
Return ONLY the complete C++ code. Do not include explanations,
markdown formatting, or code blocks.
Just the raw C++ source code that can be directly compiled.
```

B. Examples of Fine-Tuning Dataset

B.1. Example of Event Generation Strategies.

B.2. Example of Hidden Unit Test

Figure 4 is an example of the finetuning data and the corresponding unit test for verification.

Code A (Efficient): Parallel Map-Scan-Write	Code B (Inefficient): Sequential Push
<pre> 1 // 1. MAP: Count events in parallel (No Locking) 2 parlay::sequence<int> counts(N); 3 parlay::parallel_for(0, N, [&](int r) { 4 int cnt = 0; /* logic checks ... */ 5 if (valid) cnt++; 6 counts[r] = cnt; 7 }); 8 9 // 2. SCAN: Calculate offsets (Prefix Sum) 10 auto [offsets, total] = parlay::scan(counts); 11 12 // 3. WRITE: Parallel Fill (Zero Realloc/ Contention) 13 parlay::sequence<Event> evs(total); // Alloc exact size 14 parlay::parallel_for(0, N, [&](int r) { 15 int k = offsets[r]; 16 /* logic checks ... */ 17 if (valid) evs[k++] = {u, v, t}; 18 }); 19 </pre>	<pre> 1 // 1. Setup Vector (Heuristic reservation) 2 std::vector<Event> events; 3 events.reserve(2 * N * N); // May still realloc 4 5 // 2. Iterate Sequentially (Cannot Parallelize) 6 for (int r = 0; r < N; ++r) { 7 for (int c = 0; c < N; ++c) { 8 /* logic checks ... */ 9 10 // BOTTLENECK: Single thread, capacity checks, // and reallocation overhead. 11 if (valid) { 12 events.push_back({u, v, t}); 13 } 14 } 15 } 16 } 17 </pre>

Figure 3. Comparison of Event Generation Strategies. Left: Code A employs a Map-Scan-Write pattern to enable lock-free parallel writing. Right: Code B relies on sequential push_back, preventing parallelization and incurring reallocation costs.

Example Data Point from Parlay-Instruct

Instruction:

Filter even numbers from 0 to 100,000 using `parlay::delayed::filter_op` and materialize the result.

Target Code (Verified):

```

auto seq = parlay::iota(100000);
// specific delayed filter operation
auto f = parlay::delayed::filter_op(seq, [](int x) {
    return x%2==0 ? std::optional{x} : std::nullopt;
});
auto s = parlay::delayed::to_sequence(f);

```

Hidden Unit Test:

```

1 int main() {
2     // ... [setup code injected here] ...
3     assert(s.size() == 50000);
4     for(int i=0; i<50000; i++) assert(s[i] == i*2);
5     std::cout << "Passed";
6 }

```

Figure 4. A representative sample from the training corpus. Each sample includes a natural language instruction, the ground-truth parallel implementation, and an executable unit test used for verification.

C. Detailed Experimental Data

C.1. Comprehensive ParEval Benchmarks

Table 5 provides the complete breakdown of ‘Build@1’, ‘Pass@1’, and ‘Speedup@1’ metrics across commercial and open-weight models. The fine-tuned ParEVO models consistently outperform baselines in compilation rates and execution speed.

ParEVO: Synthesizing Code for Irregular Data

Model	Temp.	Code	Sched.	Build@1	Pass@1	Speedup@1		
						AM	GM w/ Out.	GM w/o Out.
Gemini-2.5-Flash	0.2	Parlay	Parlay	0.57	0.29	6.88	2.75	2.41
Gemini-2.5-Pro	0.2	Parlay	Parlay	0.84	0.54	85.34	5.47	3.17
Gemini-3-Pro	0.2	Parlay	Parlay	0.25	0.23	3.68	1.53	1.42
GPT-5 Thinking	0.2	Parlay	Parlay	0.93	0.05	0.43	1.13	1.13
Claude Opus 4.5	0.2	Parlay	Parlay	0.97	0.05	0.42	1.13	1.13
Gemini-2.5-Parlay (ParEVO)	0.2	Parlay	Parlay	0.81	0.58	107.43	6.33	3.74
Qwen3-Parlay (ParEVO)	0.2	Parlay	Parlay	0.50	0.33	5.01	1.87	1.64
DeepSeek-6.7B-Base	0.2	Parlay	Parlay	0.89	0.11	0.73	1.14	1.14
DeepSeek-Syntax	0.2	Parlay	Parlay	0.85	0.12	2.42	1.38	1.29
DeepSeek-Parlay (ParEVO)	0.2	Parlay	Parlay	0.81	0.26	126.16	3.14	1.63
Qwen2.5-Coder-32B	0.2	Parlay	Parlay	0.93	0.11	5.02	1.82	1.59
Qwen2.5-Coder-32B	0.7	Parlay	Parlay	0.93	0.18	11.80	2.65	1.98
Qwen2.5-Coder-32B-Instruct	0.2	Parlay	Parlay	0.61	0.41	7.00	2.22	1.82
Qwen3-Coder-30B-Instruct	0.2	Parlay	Parlay	0.51	0.28	5.15	1.96	1.72
DeepSeek-Coder-V2-Lite-Base	0.2	Parlay	Parlay	0.80	0.09	1.30	1.19	1.19
DeepSeek-Coder-V2-Lite-Base	0.7	Parlay	Parlay	0.92	0.14	4.74	1.71	1.59
StarCoder2-15B	0.2	Parlay	Parlay	0.80	0.27	7.65	2.02	1.62
Gemini-3-Pro	0.2	OMP	OMP	0.78	0.72	445.14	6.75	3.20
Gemini-2.5-Parlay	0.2	OMP	OMP	0.94	0.71	373.78	7.77	3.22
GPT-5 Thinking	0.2	OMP	OMP	0.95	0.06	0.60	1.15	1.15
Claude Opus 4.5	0.2	OMP	OMP	0.97	0.07	0.52	1.14	1.14
Qwen2.5-Coder-32B-Instruct	0.2	OMP	OMP	0.91	0.65	10.47	2.77	1.98
Qwen2.5-Coder-32B-Instruct	0.7	OMP	OMP	0.92	0.65	11.20	3.44	2.46
Qwen3-Coder-30B-Instruct	0.2	OMP	OMP	0.86	0.55	11.88	2.45	1.61
Qwen3-Coder-30B-Instruct	0.7	OMP	OMP	0.91	0.56	11.47	2.53	1.68
Qwen2.5-Coder-32B	0.2	OMP	OMP	0.98	0.35	10.37	2.56	1.61
Qwen2.5-Coder-32B	0.7	OMP	OMP	0.97	0.39	11.51	3.48	2.47
DeepSeek-Coder-V2-Lite-Base	0.2	OMP	OMP	0.82	0.24	5.60	2.25	2.09
DeepSeek-Coder-V2-Lite-Base	0.7	OMP	OMP	0.96	0.31	12.47	3.35	2.21
StarCoder2-15B	0.2	OMP	OMP	0.97	0.26	8.23	1.95	1.47
Gemini-3-Pro	0.2	Rust	Rayon	0.97	0.82	7.32	0.69	0.61
Qwen2.5-Coder-32B-Instruct	0.2	Rust	Rayon	0.63	0.49	4.61	0.57	0.51
Qwen2.5-Coder-32B-Instruct	0.7	Rust	Rayon	0.70	0.48	5.69	0.51	0.44
Qwen3-Rust (ParEVO)	0.2	Rust	Rayon	0.64	0.46	4.62	0.35	0.29
Qwen3-Coder-30B-Instruct	0.2	Rust	Rayon	0.61	0.50	4.57	0.46	0.38
Qwen3-Coder-30B-Instruct	0.7	Rust	Rayon	0.66	0.49	4.94	0.29	0.24
Qwen2.5-Coder-32B	0.2	Rust	Rayon	0.82	0.45	4.43	0.63	0.63
Qwen2.5-Coder-32B	0.7	Rust	Rayon	0.86	0.38	4.35	0.79	0.79
DS-Coder-V2-Lite-Base	0.2	Rust	Rayon	0.73	0.29	3.47	0.58	0.54
DS-Coder-V2-Lite-Base	0.7	Rust	Rayon	0.85	0.25	4.96	0.89	0.78
StarCoder2-15B	0.2	Rust	Rayon	0.77	0.27	2.46	0.79	0.74
StarCoder2-15B	0.7	Rust	Rayon	0.82	0.25	5.67	0.63	0.49
DS-Coder-V2-Lite-Instruct	0.2	Rust	Rayon	0.40	0.02	0.15	1.04	1.04
DS-Coder-V2-Lite-Instruct	0.7	Rust	Rayon	0.48	0.02	0.15	1.04	1.04

Table 5. Comprehensive ParEval results for commercial and local models. Shaded regions distinguish C++/ParlayLib/OMP models (Green) from Rust/Rayon models (Purple). To isolate pure parallel scaling, we also conducted an ablation removing programs with speedups $\geq 32\times$ (indicating algorithmic improvement beyond physical core counts), as denoted by the GM w/o Out. column.

C.2. Metric Breakdown by Problem Type

To understand the specific impact of fine-tuning, we visualize the shift in metrics across problem types. Figure 5 and Figure 6 demonstrate that while fine-tuning universally improves Build and Pass rates, the Speedup gains are most pronounced in the irregular graph and complex arithmetic categories. This applies to Figure 7 except that we see occasional

lower Pass rate after finetuning.

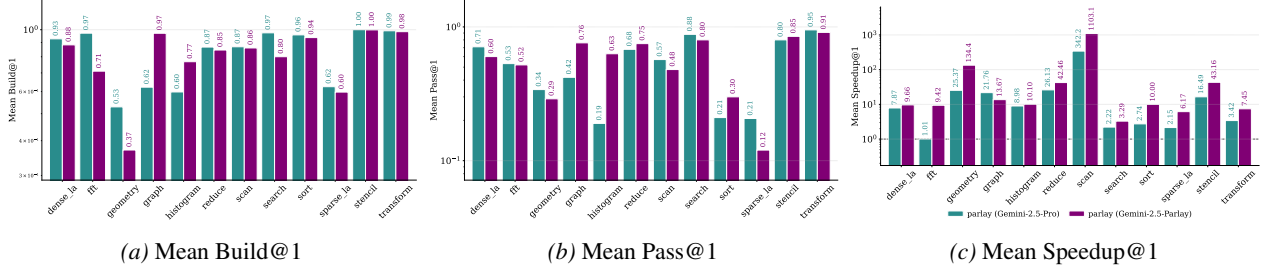


Figure 5. ParEval Metrics Comparison between Gemini-2.5-Pro and Gemini-2.5-Parlay, where the former is the base model and the latter is the base model finetuned on the Parlay-Instruct and DMOJ datasets. (a-c) highlight that fine-tuning significantly improves the model’s ability to construct valid ParlayLib code, with substantial gains in build and pass rates as well as improved running time over the base model.

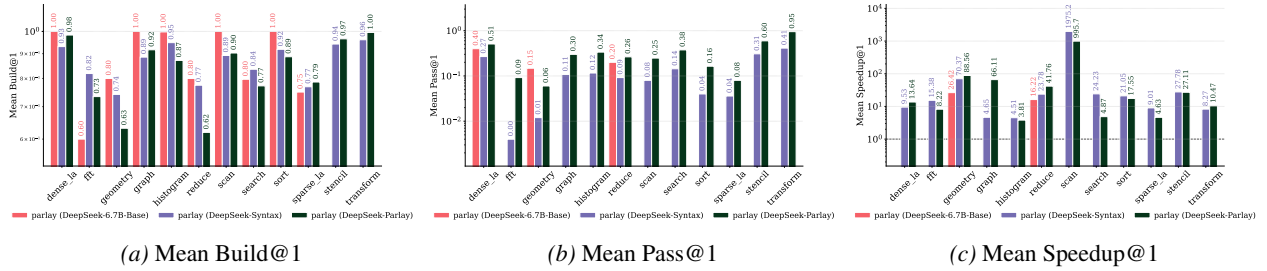


Figure 6. Impact of Fine-tuning on DeepSeek-6.7B. The fine-tuned model (DeepSeek-Parlay) shows massive gains in pass rate and speedup compared to the base model. The DeepSeek-Syntax model is the finetuned model of DeepSeek-6.7B-Base purely on ParlayLib syntax.

C.3. Comparison vs. Expert Human Baselines (PBBS & RPB)

A key contribution of this work is benchmarking against expert human code. Figure 8 compares our best generated solutions against the PBBSBench (C++) and RPB (Rust) baselines. ParEVO solutions frequently match or exceed the human baselines. Figure 8 visualizes the runtime and scalability profiles.

C.4. Case Study: The Safety vs. Performance Trade-off

A deeper analysis of the Graph Shortest Path problem reveals a subtle trade-off introduced by fine-tuning. As shown in Figure 14, the base model produces a wide variance of runtimes, occasionally hitting a very fast (but risky) solution using atomic operations. The fine-tuned ParEVO model produces highly consistent but slightly slower code, preferring safe high-level primitives (like ‘parlay::unique’) over raw memory manipulation. The detailed code samples are shown in Figure 15.

C.5. Case Study: Performance Stability on ParEval Problem 34 (Scan)

Similarly to the Shortest Path problem, we observe a distinct stabilization of performance in the fine-tuned model for ParEval Problem 34 (Scan), as shown in Figure 16. The base model’s runtime distribution Figure 16(a) is somewhat disjointed, with some runs being very slow and others faster. On the other hand, the fine-tuned model Figure 16(b) demonstrates a much tighter, more predictable runtime distribution. This consistency confirms that the fine-tuned ParEVO model systematically converges on stable and reliable parallel patterns.

C.6. Failure Modes: Geometric Hallucinations

While fine-tuning improves general syntax, it can induce “confident hallucinations” in domains with specialized logic. In the Convex Hull task (Table 6), the fine-tuned model failed by repeatedly calling a non-existent `parlay::convex.hull`

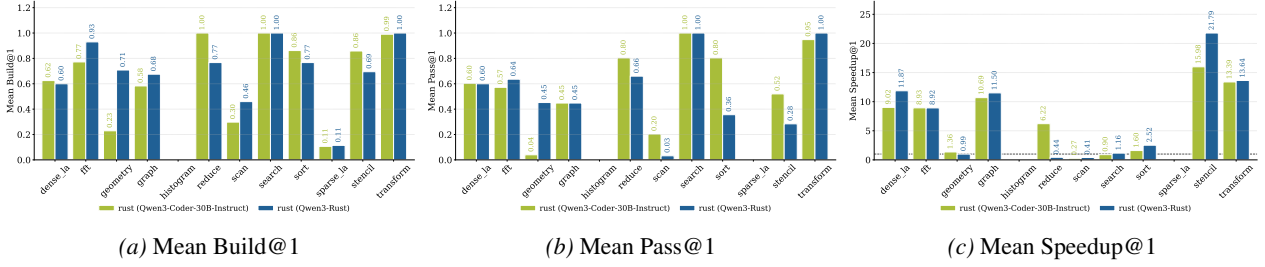


Figure 7. Impact of Fine-tuning on Qwen3-Coder-30B-A3B-Instruct. The fine-tuned model (Qwen3-Rust) shows gains in speedup compared to the base model.

function, whereas the base model attempted (and occasionally succeeded at) a manual implementation. This highlights the necessity of the ECA’s compiler-feedback loop to catch API hallucinations.

Problem Type	Model	Pass@1	Speedup@1
10_convex_hull	Gemini-2.5-Pro	0.45	1.43
10_convex_hull	DS-Parlay (ParEVO)	0.00	0.00
13_closest_pair_2d	Gemini-2.5-Pro	0.40	74.48
13_closest_pair_2d	DS-Parlay (ParEVO)	0.45	188.03

Table 6. Detailed Geometry Results. The fine-tuned model dominates in Closest Pair but hallucinates APIs in Convex Hull.

D. Prompts

ParEval Prompts For the ParEval benchmarks, we adopt the prompting specifications outlined by Nichols et al. (Nichols et al., 2024). We utilize a fixed system instruction alongside language-specific templates for C++ and Rust.

The system prompt provided to the model is as follows (see Figure 17).

Extending ParEval for Parallel Libraries Since the original ParEval dataset lacks native support for ParlayLib and Rust, we manually curated task-specific prompts to bridge this gap. These prompts preserve the original problem semantics while explicitly requesting the use of specific parallel frameworks (ParlayLib for C++ and Rayon for Rust). Figure 18 demonstrates how a standard Discrete Fourier Transform (DFT) task is adapted for both languages.

PBBSBench Prompting Strategy We employ two distinct prompting strategies for PBBSBench to evaluate the model’s ability to utilize context:

- **Concise Prompts:** These contain only the natural language problem description and the target function signature.
- **Augmented Prompts:** These extend the concise version by including definitions for necessary ParlayLib primitives, custom data structures (e.g., `Graph`), and helper utilities (e.g., `Graph_io`) defined within the PBBSBench environment.

Figure 19 illustrates an example of the concise prompting format.

RPB Prompting Strategy The prompting strategy for the RPB benchmarks relies on a composite structure. Each prompt comprises two distinct segments: (1) a context block defining Rust primitives that replicate ParlayLib functionality (e.g., `flatten`), and (2) the specific problem statement, including allowed libraries and the target function signature.

E. Fine-tuning on Low-level APIs vs. ParlayLib Token-efficiency

To assess whether the choice of parallel abstraction affects fine-tuning efficiency, we compare `deepseek-coder-6.7b-base` fine-tuned on our Parlay-Instruct dataset against an equivalent model fine-tuned on a token-matched OpenMP dataset.

The OpenMP dataset was sourced from `hpcgroup/hpc-instruct` and cleaned to match the 2.14M token budget of Parlay-Instruct, isolating core algorithmic loops to provide a dense learning signal. The filtering process consisted of:

1. Dropping translation tasks and non-native OpenMP languages (Python, Chapel, OpenCL, CUDA).
2. Retaining only samples with actual `#pragma omp` or `!$omp` directives in the solution.
3. Dropping outputs from weaker generator models (e.g., Mixtral, DBRX).
4. Deduplicating by seed, keeping the best model output per seed.
5. Trimming the longest samples to hit the 2.14M token target, retaining shorter, more focused examples.

Both models were trained with identical SFT+LoRA hyperparameters (LoRA $r = 8$, $\alpha = 16$, dropout = 0.05, max_seq_length = 2048, LR = 2×10^{-4}). Results on three representative PBBS problems are shown in Table 7.

Model	Exec. Model	Problem Name	build@1	pass@1	speedup@1
DeepSeek-Syntax	parlay	06_fft_dft	0.937	0.020	30.754
DeepSeek-Syntax	parlay	16_graph_largest_component	0.856	0.060	3.517
DeepSeek-Syntax	parlay	19_graph_shortest_path	0.919	0.080	2.187
DeepSeek-OMP	omp	06_fft_dft	1.000	0.550	23.180
DeepSeek-OMP	omp	16_graph_largest_component	1.000	0.000	0.000
DeepSeek-OMP	omp	19_graph_shortest_path	0.960	0.100	0.000

Table 7. Token-matched comparison of Parlay-Instruct vs. OpenMP fine-tuning on deepseek-coder-6.7b-base.

Under a fixed token budget, the Parlay-Instruct model achieves higher speedups on successful runs, while the OpenMP model produces more frequently compiling and passing solutions but with limited parallel gains. This suggests that high-level abstractions like ParlayLib provide a more token-efficient signal for learning parallel reasoning, whereas explicit low-level thread and memory management consumes capacity that might otherwise go toward algorithmic structure. Full PBBSBench evaluations will appear in a subsequent revision.

F. Ablation Study on Agent Architecture and Search Strategies

To rigorously evaluate the structural contributions of our evolutionary pipeline, we conducted an ablation study utilizing an open-weight local model (**Qwen3-Coder-30B-Instruct**). A mid-sized open model is employed to eliminate confounding variables associated with opaque, massive-scale models, and to ensure that any observed performance gains are directly attributable to our architectural advancements—specifically, the integration of MAP-Elites and dynamic execution feedback—rather than sheer parameter count.

The baseline model for all non-finetuned configurations is **Qwen3-Coder-30B-Instruct**, while our complete pipeline utilizes the fine-tuned **Qwen-Parlay**. We evaluate the following configurations:

- **Best-of-N**: Standard independent sampling without execution-driven feedback.
- **Best-of-N + CoT**: Independent sampling augmented with Chain-of-Thought prompting.
- **Self-Refine**: A single-lineage, text-only iterative refinement strategy.
- **ECA-base**: Our Evolutionary Coding Agent (ECA) pipeline applied to the baseline model, lacking domain-specific fine-tuning.
- **ECA_no_div**: The ECA pipeline utilizing runtime execution feedback, but with the MAP-Elites diversity archive disabled.
- **ECA-ft (Full ParEVO)**: Our proposed methodology, combining the fine-tuned **Qwen-Parlay** model with the complete, diversity-driven ECA pipeline.

Problem ID	Best-of-N	Best-of-N + CoT	Self-Refine	ECA-base	ECA_no_div	ECA-ft (Full)
coci11c1p3	0.1019	0.1018	0.3200	0.1054	0.1017	0.0297
ccc16j3	0.0026	0.0025	0.0031	0.0030	0.0026	0.0019
coci06c1p5	-	-	0.0040	0.0068	-	0.0069
coci12c1p4	-	0.0265	0.0264	0.0774	0.0267	0.0147
coci17c4p1	0.0030	0.0030	0.0030	0.0032	0.0018	0.0017
coci22c3p1	0.0027	0.0031	0.0032	0.0030	-	0.0030
crci08p1	0.0030	0.0031	0.0018	0.0033	-	0.0017

Table 8. Ablation results evaluating runtime performance (in seconds) across diverse competitive programming challenges. Problems ids refer to problems in the DMOJ dataset. A “-” indicates that the generated solution failed to pass the required unit tests, resulting in no valid runtime.

The results demonstrate that ParEVO’s primary advantage over standard iterative repair lies in its ability to escape local optima. Single-lineage refinement methods (e.g., Self-Refine and ECA_no_div) frequently converge on structurally flawed algorithms, exhaustively attempting syntactic patches without exploring alternative logic. By leveraging MAP-Elites to enforce structural diversity (e.g., varying synchronization primitives and underlying data structures), ECA-ft successfully navigates the broader search space to discover highly optimized parallel implementations where simpler pipelines fail to yield functionally correct solutions.

G. Ablation Study on Iteration Budgets and Fine-Tuning Synergy

This section investigates the combined efficacy of fine-tuning and the Evolutionary Coding Agent (ECA). We evaluate the synergy of fine-tuning alongside the ECA pipeline across various iteration budgets and diversity settings on selected DMOJ problems.

For these experiments, the base model is **Gemini-2.5-Pro**, and the fine-tuned variant is **Gemini-2.5-Parlay**. We define the following ablation configurations:

- **ECA-Iter5, ECA-Iter15, ECA-Iter30**: The baseline Gemini-2.5-Pro model utilizing the standard ECA pipeline (with a diversity archive of 5 programs) capped at 5, 15, and 30 evolutionary iterations, respectively.
- **ECA-No-Diversity**: The baseline Gemini-2.5-Pro model running for 30 iterations, but with the MAP-Elites diversity mechanism disabled.
- **ECA-Finetuned (Full Synergy)**: Our fully proposed pipeline, pairing the fine-tuned Gemini-2.5-Parlay model with the complete ECA pipeline for 30 iterations.

Problem ID	ECA-Iter5	ECA-Iter15	ECA-Iter30	ECA-No-Diversity	ECA-Finetuned
cco08p4	-	0.00753	0.00698	0.00927	0.00706
coci19c1p3	-	-	-	-	0.02424
coci11c1p3	0.02606	0.02565	0.02534	0.00822	0.02681
coci23c2p2	0.03474	0.03166	0.03130	0.03500	0.03608

Table 9. Ablation results detailing runtime performance (in seconds) on DMOJ problems. A “-” indicates that the generated solution failed to pass the requisite tests, yielding no valid runtime. Iteration configurations dictate the maximum number of evolutionary cycles permitted.

Crucially, **ECA-Finetuned** emerges as the most robust model configuration. On exceptionally complex algorithms—such as the graph problem coci19c1p3—every base-model configuration failed to generate a valid parallel solution, regardless of the iteration allowance. Only the combination of domain-specific fine-tuning and the evolutionary search successfully navigated the solution space to yield a correct, optimized implementation. Furthermore, this variant maintains highly competitive runtimes across all other evaluated problems.

Additionally, isolating the search parameters reveals two critical methodological trends:

1. **Iteration Budgets:** Expanding the evolutionary horizon consistently yields tighter, more optimized execution schedules. For instance, on `cco08p4`, a constrained budget of 5 iterations fails entirely, 15 iterations secures a valid solution, and 30 iterations discovers the most time-efficient implementation.
2. **Advantage of MAP-Elites Diversity:** Disabling the structural diversity constraints (`ECA-No-Diversity`) reliably traps the agent in local optima. Without the pressure to explore alternative concurrency primitives or data structures, the search can sometimes lead to invalid or degraded solutions.

H. Integrity of Unit Tests in Training Data Verification

Test integrity. A natural concern with LLM-generated training data is whether the model effectively grades itself with trivial tests. Our pipeline avoids this by reusing the human-authored test logic from the 593 seed programs. The mutation operators alter intermediate algorithmic logic and datatypes for diversity, but they preserve the deterministic input–output mapping of each seed. The Teacher therefore synthesizes new test code only when $\mathcal{M}_{type}$ changes the input or output type; in all other cases the original human-written unit test is reused. The vast majority of the corpus is thus evaluated against tests the LLM did not write. Additionally, we uniformly sampled 100 programs from the 13,820 tasks for manual review, confirming that reused tests matched their seed and that synthesized datatype-adapted tests preserved the original semantics. All sampled programs achieved 100% line coverage, ensuring no parallel logic escaped verification.

We additionally compiled and executed the full corpus under ThreadSanitizer (TSan). TSan flagged approximately 150 candidate races or runtime errors; manual inspection of 50 flagged cases identified no genuine data races. This is consistent with the structural properties of the code we generate: ParlayLib’s functional, lock-free data-parallel primitives, used in place of explicit thread management, make data races and deadlocks structurally unlikely.

I. Leakage and Near-Duplicate Overlap Analysis

To verify that our reported results reflect genuine generalization rather than memorization of evaluation problems, we analyze the relationship between our training corpora and the benchmarks used for evaluation.

Held-out tasks. The 700 held-out tasks in Parlay-Instruct corpus function exclusively as a validation set during fine-tuning and are not used in any of the final evaluations (PBBSBench, ParEval, RPB, DMOJ). Overlap between the training and validation splits therefore does not affect reported results.

Evaluation benchmarks. The evaluation benchmarks are disjoint from our training data. The Parlay-Instruct corpus, synthesized from ParlayLib, is used to fine-tune the C++ models, while a Rust version of DMOJ is used to fine-tune the Rust models. All base and fine-tuned models share identical pretraining data to ensure fair comparison.

N-gram overlap analysis. We performed an exact N -gram overlap analysis ($N = 10, 15$) using Jaccard similarity, comparing our fine-tuning corpora against each evaluation suite (Parlay-Instruct against ParEval, DMOJ, and PBBSBench; Rust-DMOJ against RPB). Results are summarized below.

- **ParEval and DMOJ (C++):** Negligible overlap (< 0.00002 for 10-grams). All matches consisted of standard library dependencies (e.g., `#include <algorithm>`) and generic variable assignments.
- **PBBSBench (C++):** Minimal overlap ($< 0.6\%$ for 10-grams). Manual review confirmed these matches are predominantly C++ header usage and structural `parlaylib` boilerplate required for compilation.
- **Rust Performance Benchmarks (RPB):** Near-zero leakage (0.0 for 15-grams; 0.000004 for 10-grams). The single overlapping 10-gram is standard Rust comparison boilerplate (`fn partial_cmp...`).
- **DMOJ fine-tune problems vs. DMOJ test problems:** 0.002559 for 10-grams.

Summary. The maximum structural overlap across all evaluation suites is below 0.6% , and manual inspection confirms these matches consist of syntactic boilerplate rather than solution content. We find no evidence of evaluation problem solutions appearing in the training data.

J. Semantic Alignment Example

Figure 21 is an example of the semantic alignment achieved by finetuning.

Input Prompt:

```
#include <parlay/primitives.h>
#include <complex>
/* Sort vector x of complex numbers by magnitude (ascending) using ParlayLib. */
void sortComplexByMagnitude(parlay::sequence<std::complex<double>>& x) {
```

Base Model (Gemini-2.5-Pro)

```
1 {
2     // Build Error: parlay::sort expects
3     // different comparator signature
4     parlay::sort(x, [](const std::complex<
5     double>& a, const std::complex<double>&
6     b) {
7         return std::abs(a) < std::abs(b);
8     });
9 }
```

Status: Compilation Failed (Build@1=0)

Figure 21. **Semantic Alignment Example.** The base model (top) fails to compile due to incorrect API usage and strict type definitions in the lambda. The fine-tuned model (bottom) correctly identifies `sort_inplace` and uses `auto` to handle the complex number types safely.

Fine-tuned Model (ParEVO)

```
1 {
2     // Correct: uses sort_inplace and
3     // generic auto params for complex type
4     parlay::sort_inplace(x, [](const auto&
5     a, const auto& b) {
6         return std::abs(a) < std::abs(b);
7     });
8 }
```

Status: Passed (17.5× Speedup)

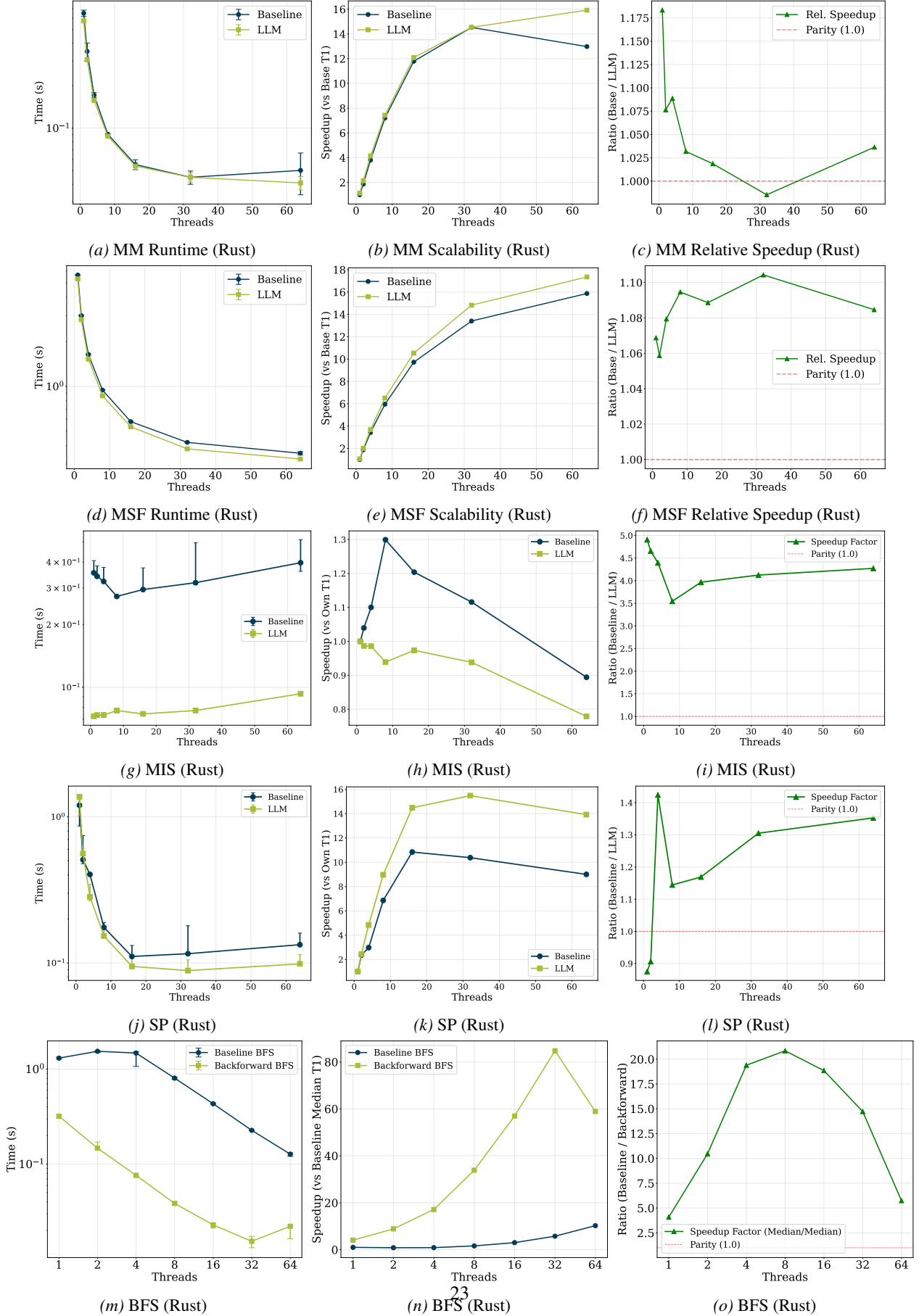


Figure 8. Runtime and Scalability comparisons against expert Rust and C++ baselines. ParEVO solutions track or beat the scalability of hand-optimized code. In (m)-(o), *BackForward BFS* specifically refers to the new BFS algorithm ParEVO generated, which uses a different method than the baseline implementation that uses multiqueue BFS.



Figure 9. Code comparison for Maximal Matching. **Left (Baseline):** Uses mixed safe/unsafe access (potential aliasing bugs) and collects results using two separate passes (red highlights). **Right (ParEVO):** Uses consistent raw pointer access to ensure memory visibility, increases block granularity to 16, and uses a single-pass unzip for result collection (green highlights).

RPB (Unsafe Baseline)

```

1 #[cfg(not(feature="openevolve"))]
2 pub fn minimum_spanning_forest(wea: &WghEdgeArray, dest: &mut
3     Vec<DefInt>) {
4     // ... Initialization ...
5     let rs: Vec<Reservation> = (0..n).map(|_| Reservation::
6         new()).collect();
7
8     // Raw pointers for some structures only
9     let _uf_ptr = &uf as *const _ as usize;
10    let _iwea_ptr = iwea.as_ptr() as usize;
11
12    let reserve = |i: usize| {
13        // Unsafe access to Edge List
14        let e = unsafe {
15            (_iwea_ptr as *mut IndexedEdge).add(i).as_mut()
16            .unwrap() // Runtime check
17        };
18        // Unsafe access to UnionFind
19        let luf = unsafe { (_uf_ptr as *mut UnionFind).as_mut
20            () .unwrap() };
21
22        e.u = luf.find(e.u as DefIntS) as DefInt;
23        e.v = luf.find(e.v as DefIntS) as DefInt;
24
25        if e.u != e.v {
26            // STANDARD INDEXING: Incurs bounds checking
27            // overhead
28            rs[e.v as usize].reserve(i as DefInt);
29            rs[e.u as usize].reserve(i as DefInt);
30            true
31        } else { false }
32    };
33
34    // ... Commit logic similar to above ...
35    (0..iwea.len()).spec_for(reserve, commit, ...);
36 }
    
```

ParEVO (Optimized)

```

1 #[cfg(feature="openevolve")]
2 pub fn minimum_spanning_forest(wea: &WghEdgeArray, dest: &mut
3     Vec<DefInt>) {
4     // ... Initialization ...
5     let rs: Vec<Reservation> = (0..n).into_par_iter().map(|_|
6         Reservation::new()).collect();
7
8     // Raw pointers for EVERYTHING (including Reservation
9     // array)
10    let _rs_ptr = rs.as_ptr() as usize;
11    let _uf_ptr = &uf as *const _ as usize;
12    let _iwea_ptr = iwea.as_ptr() as usize;
13
14    let reserve = |i: usize| {
15        unsafe {
16            // UNCHECKED UNWRAP: Eliminates null checks
17            let e = (_iwea_ptr as *mut IndexedEdge).add(i)
18                .as_mut() .unwrap_unchecked();
19            let luf = (_uf_ptr as *mut UnionFind).as_mut()
20                .unwrap_unchecked();
21
22            e.u = luf.find(e.u as DefIntS) as DefInt;
23            e.v = luf.find(e.v as DefIntS) as DefInt;
24
25            if e.u != e.v {
26                // POINTER ARITHMETIC: Eliminates bounds
27                // checks
28                let rv = (_rs_ptr as *const Reservation)
29                    .add(e.v as usize)
30                    .as_ref().unwrap_unchecked();
31                let ru = (_rs_ptr as *const Reservation)
32                    .add(e.u as usize)
33                    .as_ref().unwrap_unchecked();
34
35                rv.reserve(i as DefInt);
36                ru.reserve(i as DefInt);
37            }
38            true
39        }
40    };
41
42    (0..iwea.len()).spec_for(reserve, commit, ...);
43 }
    
```

Figure 10. Code comparison for Minimum Spanning Forest (MSF). **Left (Baseline):** Uses standard indexing for the reservation array (incurring bounds checks) and standard `unwrap()` (incurring branch checks), highlighted in red. **Right (ParEVO):** Adopts a “Maximal Unsafe” strategy, converting all data structures to raw pointers. It uses `unwrap_unchecked()` and pointer arithmetic (`.add()`) to eliminate all runtime safety checks, highlighted in green. This relies on the assumption that edge indices are always valid, allowing ParEVO to trade runtime safety checks for improved performance.

RPB (Unsafe Baseline)

```

1 #[cfg(not(feature = "openevolve"))]
2 pub fn maximal_independent_set(g: &Graph) -> Vec<u8> {
3     let n = g.n;
4     // UNSAFE: Standard Vec used for concurrent access
5     let flags: Vec<u8> = (0..n).into_par_iter()
6         .map(|_| 0) .collect(); // Standard allocation
7     let flags_ptr = flags.as_ptr() as usize;
8
9     let reserve = |i: usize, s: &mut MISState| -> bool {
10         s.flag = 1;
11         let v = g.index(i);
12         for j in 0..v.degree {
13             let ngh = v.neighbors[j] as usize;
14             if ngh < i {
15                 // DATA RACE: Reading mutable memory without
16                 // Compiler may optimize incorrectly;
17                 // Undefined Behavior
18                 let f = flags[ngh];
19                 if f == 1 { s.flag = 2; return true; }
20                 else if f == 0 { s.flag = 0; }
21             }
22             true
23         };
24
25         let commit = |i: usize, s: &mut MISState| -> bool {
26             // UNSAFE POINTER WRITE: Bypassing borrow checker
27             unsafe { (flags_ptr as *mut u8).add(i)
28                 .write(s.flag); }
29             s.flag > 0
30         };
31
32         (0..n).stateful_spec_for(
33             reserve, commit, MISState { flag: 0 },
34             20, Some(64), Some(256) // Small granularity
35         ).expect("failed speculative for");
36
37         return flags;
38     }
    
```

ParEVO (Optimized)

```

1 #[cfg(feature = "openevolve")]
2 pub fn maximal_independent_set(g: &Graph) -> Vec<u8> {
3     let n = g.n;
4     // PARALLEL ATOMICS: Safe concurrent access
5     let flags: Vec<AtomicU8> = (0..n).into_par_iter()
6         .map(|_| AtomicU8::new(0)) .collect();
7     let flags_slice = &flags[..];
8
9     let reserve = |i: usize, s: &mut MISState| -> bool {
10         let v = g.index(i);
11         let mut waiting = false;
12         for &ngh in v.neighbors {
13             let ngh = ngh as usize;
14             if ngh < i {
15                 // SAFE ATOMIC LOAD: Correct synchronization
16                 let f = unsafe {
17                     flags_slice.get_unchecked(ngh)
18                 }
19                 .load(Relaxed);
20                 if f == 1 { s.flag = 2; return true; }
21                 if f == 0 { waiting = true; }
22             }
23             s.flag = if waiting { 0 } else { 1 };
24             true
25         };
26
27         let commit = |i: usize, s: &mut MISState| -> bool {
28             if s.flag > 0 {
29                 // SAFE ATOMIC STORE
30                 unsafe { flags_slice.get_unchecked(i)
31                     .store(s.flag, Relaxed); }
32             }
33             true
34         } else { false };
35
36         (0..n).stateful_spec_for(
37             reserve, commit, MISState { flag: 0 },
38             256, None, None // Larger granularity
39         ).expect("failed speculative for");
40
41         // ZERO-COPY TRANSFORMATION: AtomicU8 -> u8
42         unsafe {
43             let mut v = std::mem::ManuallyDrop::new(flags);
44             Vec::from_raw_parts(v.as_mut_ptr() as *mut u8, v.len()
45                 (), v.capacity())
46         }
47     }
    
```

Figure 11. Code comparison for Maximal Independent Set (MIS). **Left (Baseline):** Uses unsafe standard `Vec<u8>` (red), causing undefined behavior (data races) during reads and writing via raw pointers. It uses a small block size (20). **Right (ParEVO):** Uses `Vec<AtomicU8>` (green) for correct synchronization using Relaxed ordering. It optimizes throughput with a larger block size (256) and employs a zero-copy cast to convert the atomic vector back to a standard vector at the end.

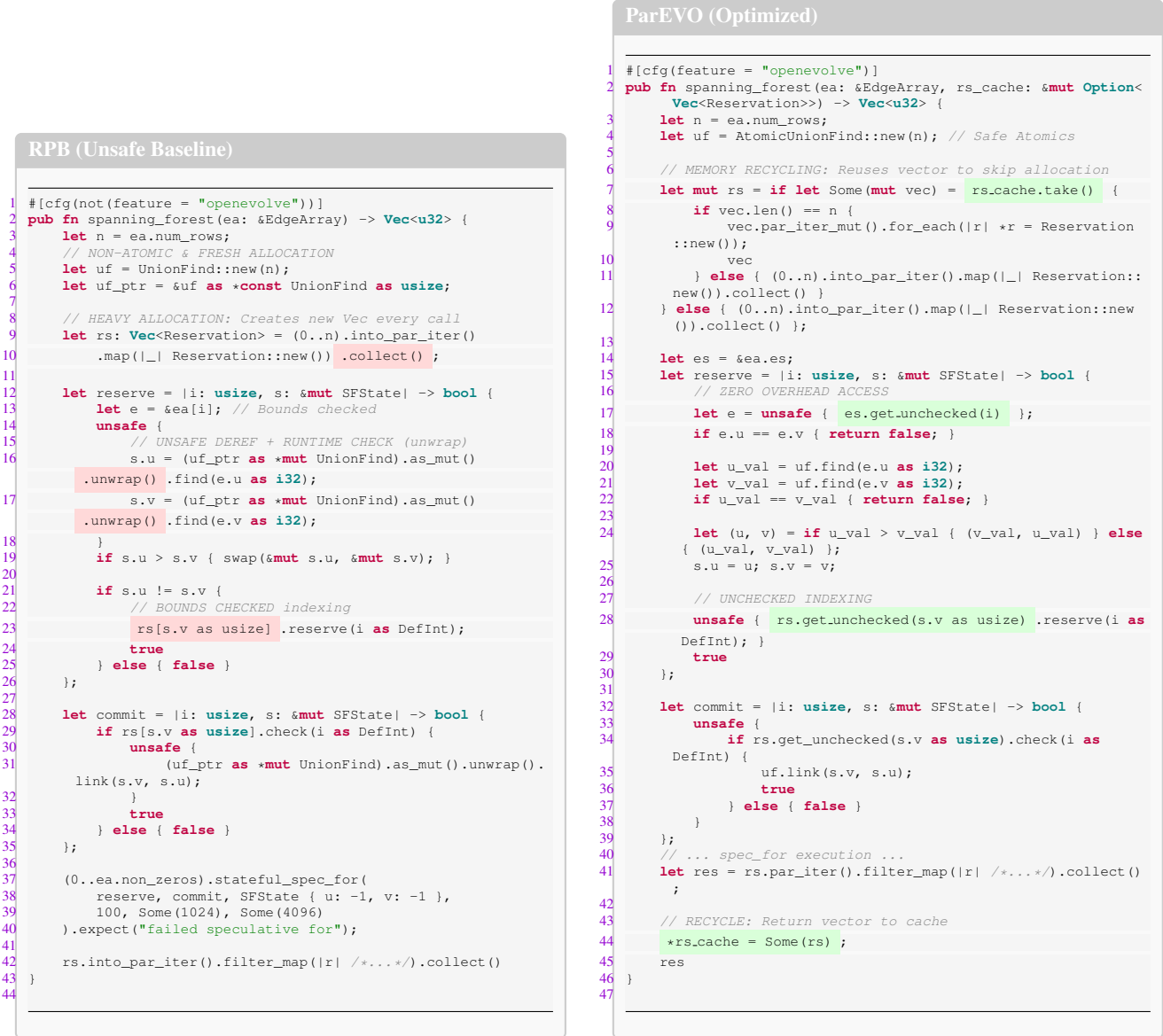


Figure 12. Code comparison for Spanning Forest. **Left (Baseline):** Performs a fresh allocation for the reservation array on every call (red) and uses checked indexing/unwrapping inside the hot loop. **Right (ParEVO):** Implements a memory recycling mechanism via `rs_cache` (green) to reuse the large reservation vector across calls. It also employs `get_unchecked` and `AtomicUnionFind` to eliminate bounds checking and pointer dereference overheads.



Figure 13. Code comparison for BFS. **Left (Baseline):** Uses a standard asynchronous approach where every edge relaxation requires a CAS loop and a queue push (red), leading to high contention on scale-free graphs. **Right (ParEVO):** Implements Direction-Optimizing BFS (Ligra-style). It dynamically switches between "Push" (Sparse) and "Pull" (Dense) modes based on the frontier density (green), drastically reducing edge checks during the heavy middle levels of the traversal.

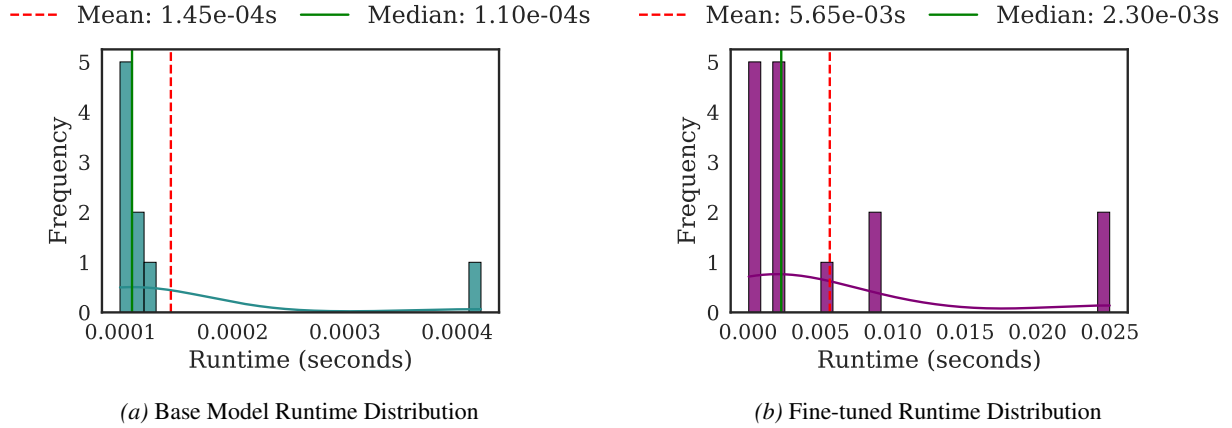


Figure 14. Runtime Histograms for Graph Shortest Path. The fine-tuned model (b) exhibits tighter variance (reliability) but a higher median runtime due to overhead from safety-focused primitives.



Figure 15. Code comparison for Shortest Path (Problem 19 in ParEval). **Left (Baseline):** Effectively uses `std::atomic` and Compare-and-Swap (CAS) to manage visitation state in parallel, resulting in a significantly faster runtime. **Right (Finetuned):** Chooses a high-overhead initialization step (copying the adjacency matrix via `tabulate`) and falls back to sequential logic for the queue update loop (red), causing $O(N^2)$ startup cost and serialization bottlenecks. Nonetheless, it shows heavier abstraction usage.

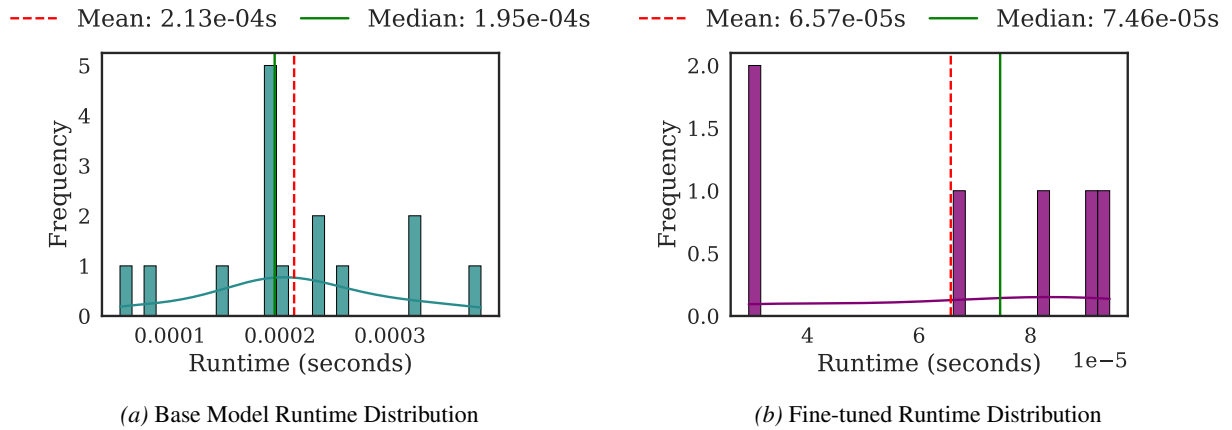


Figure 16. Runtime Histograms for ParEval Problem 34 (Scan). The fine-tuned model (b) exhibits tighter variance and highly predictable performance compared to the wider distribution of the base model (a).

ParEval Prompting Specification

System Instruction:

Fixed instruction prepended to all queries.

You are a **helpful** coding assistant.

You are helping a programmer write a C++ function. Write the body of the function and put it in a markdown code block.

Requirements:

- **DO NOT WRITE ANY COMMENTS OR EXPLANATIONS** in the code!!! Generate **PURE** code!!!
- Before you return the code, make sure to **remove any comments or explanations** that you may have added.

C++ User Template:

Complete the C++ function {function_name}. Only write the body of the function {function_name}.

```
```cpp
{prompt}
```
```

Rust User Template:

Complete the Rust function {function_name}. Only write the body of the function {function_name}.

```
```Rust
{prompt}
```
```

Figure 17. The prompting strategy adopted from the ParEval paper (Nichols et al., 2024). The templates include specific placeholders (function.name, prompt) populated dynamically during evaluation.

ParEval Extension Examples

C++ Prompt (ParlayLib):

```
/* Compute the discrete fourier transform of x. Store the result in output.
   Use ParlayLib to compute in parallel.
   Example:
       input: [1, 4, 9, 16]
       output: [30+0i, -8-12i, -10-0i, -8+12i]
*/
void dft(parlay::sequence<double> const& x,
        parlay::sequence<std::complex<double>> &output) {
```

Rust Prompt (Rayon):

```
/* Compute the discrete fourier transform of x. Store the result in output.
   Use Rust Rayon to compute in parallel.
   Example:
       input: [1, 4, 9, 16]
       output: [30+0i, -8-12i, -10-0i, -8+12i]
*/
pub fn dft(x: &[f64], output: &mut [num_complex::Complex<f64>]) {
```

Figure 18. Representative examples of our manual extensions to the ParEval dataset. The prompts are tailored to enforce specific parallel backends while maintaining identical input/output specifications.

PBBSBench Strategy: Concise Prompt

System Instruction:

You are an expert C++ programmer with extensive experience in parallel programming. Write a parallel {} procedure in C++ that is correct and is the fastest parallel {} program you can generate. Return the code between '// --- Start of file:' and '// --- End of file:' markers.

User Input (Example: Maximal Independent Set):

Returns a maximal independent set for an undirected graph. Use ParlayLib to compute in parallel.

```
#include "common/graph.h"

using vertexId = uint;
using edgeId = uint;
using Graph = graph<vertexId,edgeId>;

parlay::sequence<char> maximalIndependentSet(Graph const &G);
```

Figure 19. An example of the *Concise* prompt formulation for the PBBSBench maximalIndependentSet task. The model is provided with the function signature and a request to use ParlayLib, but implementation details of the Graph structure are omitted.

RPB Prompt Structure

Part 1: Context (Excerpt of ParlayLib-Rust Primitives)

The prompt begins by providing the full suite of helper functions (truncated here for brevity).

```
// Here are the primitives you may use
// ... [Full list of primitives omitted] ...

/* ----- Flatten ----- */
pub fn flatten<T>(arr: &[Vec<T>], dest: &mut Vec<T>)
where T: Copy + Send + Sync + Default {
    // ... implementation details ...
}

// ... [Additional primitives like scan, reduce, etc.] ...
```

Part 2: Task Definition & Signature

The specific algorithm request follows the context.

```
// Given an undirected graph, return a maximal independent set (MIS).
// The input graph can be in any format.
// The code cannot reorder the graph for locality.
// The output must be a sequence of vertices in the MIS (order irrelevant).

#[cfg(feature = "AW_safe")]
use std::sync::atomic::{AtomicU8, Ordering::Relaxed};
use rayon::prelude::*;
use pbbs::common::graph::Graph;

#[path="../../common/spec_for.rs"] mod spec_for;
use spec_for::StatefulSpecFor;

#[derive(Clone)]
struct MISState {
    flag: u8,
}

pub fn maximal_independent_set(g: &Graph) -> Vec<u8> {
    // LLM_OUTPUT_HERE
}
```

Figure 20. An example of the RPB prompting template. We inject the full set of parallel primitive definitions (represented by the flatten excerpt in Part 1) prior to the specific task instructions (Part 2) to ground the model in the available Rust-ParlayLib equivalence layer.